

IGPC-MSOS: A knowledge-preserving transfer learning framework with dynamic mode-switching for handling concept drift in network intrusion detection systems

Methaq A. Shyaa^a, Noor Farizah Ibrahim^a, Zurinahni Binti Zainol^{a,b},
Rosni Abdullah^a, Mohammed Anbar^c, Laith Alzubaidi^{d,*}

^a School of Computer Sciences, Universiti Sains Malaysia USM Gelugor, 11800, Pulau Penang, Malaysia

^b School of Computing and Informatics, Albukhary International University, Alor Setar, 05200, Kedah, Malaysia

^c Cybersecurity Research Centre(CYRES), Universiti Sains Malaysia (USM), 11800, Penang, Malaysia

^d School of Mechanical Medical and Process Engineering, Queensland University of Technology, 4000, Brisbane, QLD, Australia

ARTICLE INFO

Keywords:

Concept drift
Intrusion detection systems
Transfer learning
Mode switching

ABSTRACT

The rapid evolution of cyber threats poses significant challenges to Intrusion Detection Systems (IDS), particularly in dynamic environments affected by concept drift, where shifting attack behaviors degrade long-term detection performance. Existing adaptive IDS solutions often remain limited by fragmented drift-handling mechanisms, weak knowledge retention, and insufficient integration of complementary learning strategies, leaving exploitable blind spots. This paper introduces a unified adaptive IDS framework based on a mode-switching architecture that integrates Online Sequential Extreme Learning Machine (OSELN), Feature-Adaptive OSELN (FA-OSELN), and Knowledge-Preserving OSELN (KP-OSELN). The proposed Incremental Genetic Programming Combiner with Mode-Switching Online Sequential (IGPC-MSOS) method dynamically selects the most effective operational mode according to detected drift patterns and real-time performance feedback. Experimental evaluations across five benchmark datasets demonstrate that IGPC-MSOS consistently achieves 96%–100% recall, delivers competitive or superior F1-scores (0.96–0.9995), and reduces inference latency compared to the State-of-the-Art Approaches. These results confirm the strong adaptability, robustness, and real-time suitability of the proposed approach for intrusion detection in evolving and high-throughput network environments.

1. Introduction

The increasing sophistication of cyber threats poses serious challenges for traditional Intrusion Detection Systems (IDSs), especially in dynamic, non-stationary network environments. Modern attack strategies exploit the static nature of conventional Intrusion Detection Systems (IDS) [1], resulting in decreased detection accuracy, increased false alarms, and poor adaptability to evolving threats. One of the primary reasons for this degradation is concept drift [2], a phenomenon in which the statistical properties of the input data change over time due to shifts in attack behavior, feature relevance, or benign traffic patterns [3–5].

Concept drift in cybersecurity manifests in two dominant forms: real drift, which reflects changes in the underlying data distribution (e.g., new attack variants), and feature drift, representing shifts in feature importance or usage patterns [6–8]. The inability of static models to adapt to these variations undermines the long-term reliability of IDS in real-

world deployments. Furthermore, adaptive IDS approaches often focus on specific drift types or limited learning scopes, lacking generalizability across dynamic operational conditions.

Although various ensemble learning methods and deep neural model techniques have been explored to address this problem, current solutions exhibit significant limitations. These include an overreliance on fixed model architectures, insufficient mechanisms for knowledge retention or transfer, and a lack of unified frameworks capable of addressing multiple drift patterns simultaneously [4,9–11]. Moreover, frequent model retraining or full replacements lead to substantial computational overhead, making real-time deployment infeasible for high-throughput systems.

To overcome these challenges, we propose a novel adaptive IDS framework termed IGPC-MSOS (Incremental Genetic Programming Combiner with Mode-Switching Online Sequential Learning). Our approach introduces a dynamic architecture capable of switching among

* Corresponding author.

E-mail addresses: methaqabdullah@student.usm.my (M.A. Shyaa), nfarizah@usm.my (N.F. Ibrahim), zurinahni.zainol@aiu.edu.my (Z.B. Zainol), rosni@usm.my (R. Abdullah), anbar@cyres.usm.my (M. Anbar), l.alzubaidi@qut.edu.au (L. Alzubaidi).

<https://doi.org/10.1016/j.knosys.2026.115361>

Received 30 June 2025; Received in revised form 10 January 2026; Accepted 16 January 2026

Available online 20 January 2026

0950-7051/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

three operational classifiers. Online Sequential Extreme Learning Machine (OSELN), Feature-Adaptive OSELN (FA-OSELN), and Knowledge-Preserving OSELN (KP-OSELN). These classifiers are integrated via a genetic programming-based combiner, which leverages both transfer learning and knowledge preservation to intelligently select the optimal mode based on the detected drift type.

While our previously proposed IGPC framework [12] demonstrated promising results for intrusion detection in dynamic environments, it was limited by its single-mode operational architecture that handled concept drift variants in isolation. The framework's specialized variants (GPC-OS, GPC-FOS, GPC-KOS) each excelled in specific drift scenarios. However, they lacked an integrated mechanism to dynamically switch between operational modes when faced with multiple types of drift simultaneously. Additionally, knowledge preservation occurred within individual model boundaries, hindering cross-drift knowledge transfer, which is essential in real-world network environments where drift patterns frequently overlap and coexist. To address these limitations, we propose IGPC-MSOS, which extends the original framework with a unified mode-switching mechanism and enhanced knowledge transfer capabilities that intelligently select the optimal operational classifier based on detected drift characteristics.

To the best of our knowledge, this is the first work that integrates (1) dynamic mode-switching capabilities triggered by both explicit drift detection and performance monitoring, (2) specialized operational modes with transfer learning for feature adaptation, and (3) memory-enhanced knowledge preservation mechanisms within a unified genetic programming framework. While individual components build upon existing techniques, the integration creates a comprehensive, adaptable architecture that uniquely addresses the full spectrum of concept drift patterns in network security environments. While previous approaches have explored individual adaptation mechanisms, our work represents the first comprehensive integration of specialized operational modes with intelligent switching capabilities in a unified framework. The novelty lies not merely in combining existing techniques, but in the synergistic interaction among mode-switching triggers, transfer learning, and knowledge preservation that enables adaptation to multiple concurrent drift patterns—a capability absent in existing systems. The main contributions of this paper are as follows:

- The Incremental Genetic Programming Combiner with Mode-Switching Online Sequential (IGPC-MSOS) framework, which, for the first time, unifies three specialized operational modes within a single adaptive IDS: (i) Online Sequential Extreme Learning Machine (OSELN) for standard incremental learning, (ii) Feature Adaptive OSELN (FA-OSELN) employing transfer learning for feature evolution, and (iii) Knowledge Preserving OSELN (KP-OSELN) utilizing memory mechanisms to respond effectively to recurring drift patterns.
- We introduce an intelligent mode-switching architecture with integrated knowledge preservation, dynamically selecting operational modes based on explicit drift detection signals, and continuous performance monitoring. This dual-trigger strategy incorporates mathematically formulated transfer mappings in FA-OSELN and memory-enhanced structures in KP-OSELN, enabling cross-mode knowledge retention. As a result, the framework achieves robust adaptation to novel, gradual, and recurring attack patterns without requiring manual intervention.
- We repurpose genetic programming as a dynamic evolutionary combiner, where ensemble structures are not statically fixed but evolve continuously in response to concept drift conditions and performance metrics. This provides a novel methodological synergy in IDS research by coupling adaptive mode-switching with evolutionary ensemble optimization.
- We provide a comprehensive empirical evaluation on five benchmark datasets (KDD99, CICIDS2017, ISCX2012, CSE-CIC-IDS2018 and HIKARI2021). The assessment rigorously compares IGPC-MSOS

against three specialized variants of our earlier IGPC framework (GPC-OS, GPC-FOS, GPC-KOS) [12] and two new state-of-the-art baselines: FlowTransformer [63] and the Adaptive Tree-like Neural Network (ATNN) [5]. An ablation study further demonstrates the significant performance advantages of IGPC-MSOS during concept drift, validating both its theoretical and practical impact.

The remainder of this paper is organized as follows. Section 2 reviews related work in adaptive IDS and concept drift handling. Section 3 presents the methodology of our mode-switching approach. Section 4 details the experimental setup and performance evaluation. Section 5 discusses the results and implications for network security. Finally, Section 6 concludes with a summary of contributions and directions for future research.

2. Literature survey

2.1. Concept drift aware IDS methods

The increasing reliance on data-driven intrusion detection systems necessitates models that can adapt to concept drift, a phenomenon in which the statistical properties of the data change over time. This subsection provides a systematic analysis of recent IDS approaches, revealing critical limitations across detection paradigms and knowledge preservation mechanisms.

Recent ensemble methodologies demonstrate promising capabilities, but exhibit pattern-specific limitations. The Adaptive Random Forest with Hoeffding bounds [1] achieves high accuracy on CIC-IDS 2018 but lacks mechanisms for incremental or recurring drift patterns. The ADWIN-SVM integration [15] effectively addresses the sudden drift in KDD Cup 99 and CICIDS 2017 but shows marked degradation with gradual evolution. GP-based ensemble learning [13] integrates multiple drift detectors (ADWIN, STEP, DDM) but suffers from a static combiner architecture, creating adaptation bottlenecks. Our previously proposed IGPC framework [12] introduced specialized variants for different drift types but lacked a unified mechanism to dynamically switch between operational modes, limiting its efficacy in environments with multiple concurrent drift patterns. The K-means clustering ensemble [16] achieves high accuracy but requires manual threshold tuning and is primarily used to manage gradual drift.

Deep learning approaches offer enhanced representational capacity but face challenges related to interpretability and computational efficiency. The INSOMNIA framework [17] employs DNNs with uncertainty sampling but lacks explicit drift-type categorization mechanisms. Multi-agent CNN-LSTM architectures [18] demonstrate continuous adaptation through knowledge distillation but incur substantial computational overhead. LSTM-based approaches with multi-head attention [6] improve recall rates but exhibit limited adaptability to diverse drift patterns and high computational requirements.

The preservation and transfer of knowledge represent a critical gap in multiple approaches. The AXGB Ensemble [19] rely on complete model replacement, discarding historical knowledge. Adaptive Random Forest variants [1,20] offer high accuracy but lack knowledge preservation mechanisms and show restricted evaluation scope.

Infrastructure integration and specialized approaches demonstrate domain-specific effectiveness but sacrifice generalizability. SIEM-based drift detection [10] provides real-time capabilities but relies on manual rule updates. IoT-specific DNN adaptations [21] lack robust mechanisms for differentiating security-relevant drift. The Optimized Isolation Forest (OIFIDS) [22] acknowledges knowledge preservation requirements but demonstrates persistent gaps in comprehensive solutions.

Self-supervised and reinforcement learning innovations show promise but remain limited in scope. The ReCDA framework [4] employs drift-aware perturbations that reduce labeling requirements but has not been tested across incremental scenarios. Ensemble Adaptive

Online Machine Learning [23] achieves high accuracy but fundamentally lacks explicit drift detection mechanisms.

Heterogeneous and distributed architectures provide robustness but introduce coordination challenges. Heterogeneous ensemble classifiers [24] achieve high accuracy but incur computational costs and lack feature drift awareness.

Contrastive learning and secure adaptive approaches offer specialized capabilities but suffer from limitations. The contrastive embedding framework [3] enables adaptation to feature drift but lacks pattern-specific mechanisms and interpretability. The Multilayer Perceptron Gradient Decision Tree [25] with Secure Adaptive Windowing enhances malware detection but offers minimal coverage beyond gradual drift.

Ensemble weighting and aggregation strategies demonstrate adaptability but overlook critical aspects. The Probability-Weighted Probability Averaging Ensemble [14] shows robust performance but overlooks feature-level relevance shifts. The uncertainty-fusion model [26] effectively handles sudden and gradual drift but exhibits weakness against incremental variations. The MSANA framework [27] integrates multiple adaptation mechanisms but remains bottlenecked by the limitations of base learning.

Alert aggregation and autoencoder-based approaches offer specialized capabilities but exhibit architectural rigidity. The teacher-student autoencoder ADTCD [28] reduces false positives but lacks adaptability to incremental drift patterns. CLEAR for real-time alert aggregation [29] offers near real-time processing but lacks model-level adaptation capabilities.

2.2. Concept drift aware methods

Beyond security-specific applications, the broader literature on concept drift adaptation offers valuable insights into detection and response mechanisms. Analysis reveals persistent limitations in handling complex drift patterns across diverse domains. Domain adaptation and continuous learning approaches demonstrate knowledge transfer capabilities but face architectural constraints. The Concept Drift Domain Adaptation (CDDA) [30] transfers knowledge between windows but its fixed design constrains adaptability to compound drift patterns. The Adaptive Continuous Kernel Learning (ACKL) [31] adapts via kernel adjustment but exhibits pronounced parameter sensitivity, leading to up to 15% performance degradation. The dual-layer variable sliding window approach [32] detects sudden and gradual drift but requires extensive parameter tuning.

Hybrid ensemble approaches integrate detection and adaptation mechanisms but face scalability challenges. The Hybrid Block-Based Ensemble (HBBE) [33] handles sudden and gradual drifts without requiring complete retraining, but it exhibits limitations with recurring concepts and growth in memory consumption. The heterogeneous ensemble with Q-statistic diversity measures [34] shows competitive performance but struggles with recurring and blip drifts.

Deep learning adaptation strategies offer model plasticity but sacrifice efficiency and interpretability. The adaptive windowing-based GRU [35] employs elastic weight consolidation but necessitates retraining new layers after drift detection. The hybrid deep learning approach [36], which combines LSTMs and RNNs, relies on complete retraining, making it vulnerable to catastrophic forgetting. A notable recent work, the Adaptive Tree-like Neural Network (ATNN) [5], mitigates catastrophic forgetting through tree-structured incremental learning and Fisher-based regularization to preserve historical knowledge. Despite supporting both sudden and gradual drift, ATNN lacks transfer learning and mode-switching capabilities, and its recursive model expansion incurs substantial computational overhead, limiting its viability in time-sensitive applications.

Specialized approaches for recurring drift demonstrate pattern-specific effectiveness but sacrifice general adaptability. The Recurrent Adaptive Classifier Ensemble (RACE) [37] effectively reuses historical patterns for recurring drift but exhibits marked deficiencies with sud-

den or gradual patterns. The Probabilistic Exact Adaptive Random Forest (PEARL) [38] and Nacre [39] offer sophisticated recurring concept mechanisms but demonstrate reduced effectiveness with other drift categories.

Diversity-driven ensemble strategies offer robustness but incur substantial resource requirements. The diversity framework CDCMS [40] capitalizes on model space clustering but demonstrates high memory requirements and struggles with severe gradual drift scenarios. Statistical approaches including the Kappa Updated Ensemble (KUE) [41] combine online and block-based learning but exhibit delayed response to sudden drift events.

Loss monitoring and ensemble learning approaches provide adaptation signals but often require extensive retraining. The Broad Ensemble Learning System (BELS) [42] utilises mini-chunks and ensemble layers but lacks explicit strategies for adapting to different drift patterns. The Adaptive Random Tree ensemble [11] effectively handles sudden and gradual drift via dynamic model updates; however, it lacks knowledge preservation and transfer learning, limiting its scalability in evolving environments. Similarly, the elastic gradient boosting decision tree (eGBDT) [43] detects drift through loss monitoring but lacks mechanisms tailored to specific drift type patterns, necessitating complete model retraining.

Virtual and real drift-separation approaches offer differentiated handling, but struggle with mixed patterns of data. The online Gaussian Mixture Model with Noise Filter (OGMMF-VRD) [44] addresses different drift types through separate mechanisms but critically struggles with gradual and mixed patterns. Class-imbalance integration approaches, including CIDD ADODNN [45] and cost-sensitive ensemble CSDS [46], address minority-class detection but sacrifice adaptation specificity.

Dynamic ensemble selection and streaming approaches offer instance-level adaptation, but face computational complexity. The Dynamic Updated Ensemble (DUE) [47] learns non-stationary flows that are unbalanced, but it shows limited ability to handle incremental drift patterns. The Mixed Incremental Semi-Supervised Extreme Learning Machine (MIS-ELM) [48] offers rapid updates but shows limited adaptability to gradual patterns. The SCHCDOE [49] streaming data classification approach SCHCDOE [49] categorises data states, but incurs computational overhead, which restricts real-time applications.

Advanced ensemble and optimization approaches demonstrate sophistication but exhibit specific limitations. The Dynamic Ensemble Selection for Imbalanced Data (DES-ICD) [50] features instance-level selection but struggles with sudden changes due to distance-based limitations. ElStream [51] demonstrates versatility but lacks explicit mechanisms for drift detection. The Adaptive Incremental-Decremental SVM (AIDSVM) [52] demonstrates excellent adaptation to sudden drift but shows suboptimal handling of gradual and recurring patterns. These approaches collectively demonstrate the persistent challenges in creating unified frameworks that can handle diverse manifestations of drift while maintaining operational efficiency.

Table 1 presents a systematic comparison between IGPC-MSOS and state-of-the-art approaches. The analysis evaluates critical capabilities essential for comprehensive concept drift handling: (1) dynamic ensemble learning, (2) transfer learning, (3) knowledge preservation, (4) mode switching, (5) feature drift, (6) concept drift handling diversity, and (7) implementation distinction. The results show that the existing approaches exhibit significant limitations: While ARF [1], PWPAE [14], and MSANA [27] implement dynamic ensemble learning, they lack transfer learning capabilities. Similarly, IGPC [12] supports transfer learning but operates in single-mode only. In particular, IGPC-MSOS uniquely provides comprehensive support in handling the spectrum of concept drift patterns. This comprehensive integration addresses the critical gap in unified frameworks for dynamic network environments, where adaptive response to evolving attack patterns is paramount.

Table 1

Novelty comparison with closest related works. (Legend: D = Dynamic Ensemble Learning; T = Transfer Learning; K = Knowledge Preservation; M = Mode Switching; F = Feature Drift; CDH = Concept Drift Handling (S: Sudden, G: Gradual, I: Incremental, R: Recurring)).

Method	D	T	K	M	F	CDH	Key Distinction
IDS Methods with Dynamic Ensemble Learning							
ARF [1]	✓	×	×	×	×	S,G	Hoeffding tree ensemble for sudden/gradual drift
PWPAE [14]	✓	×	✓	×	×	S,G	Probability weighting; neglects feature drift
GP Ensemble [13]	✓	×	×	×	×	S	Genetic programming dynamic combiner
Online ML [23]	✓	×	×	×	×	S,G	Ensemble adaptation without explicit drift detection
MSANA [27]	✓	×	✓	×	✓	S,G	Multiple adaptations; base learner limitations
ARF [20]	✓	×	×	×	×	S	Model replacement discards prior knowledge
General Methods with Dynamic Ensemble Learning							
HBBE [33]	✓	×	✓	×	×	S,G	Block-based learning; scaling issues
DDCW [34]	✓	×	×	×	×	S	Q-statistic diversity measures
KUE [41]	✓	×	×	×	×	S,G	Kappa-based online/block learning
DUE [47]	✓	×	×	×	×	S,G	Designed for imbalanced streams
Adaptive Random Tree [11]	✓	×	×	×	×	S,G	Adaptive tree ensemble; class imbalance handling
SCHDOE [49]	✓	×	×	×	×	S,G	State categorization with overhead
DES-ICD [50]	✓	×	×	×	×	S	Instance-level dynamic selection
Selected Non-DEL Methods							
LSTM [6]	×	×	✓	×	×	S,G	Sliding window; limited adaptation
ReCDA [4]	×	×	✓	×	✓	S,G	No support for knowledge transfer
SIEM-ADWIN [10]	×	×	×	×	×	S	Manual rule dependency
ATNN [5]	×	×	✓	×	×	S,G	Tree expansion; Fisher-based retention
ADWIN-SVM [15]	×	×	×	×	×	S	No knowledge retention/transfer
IGPC [12]	×	✓	✓	×	✓	S,G,R	Single-mode operation limitation
IGPC-MSOS (Ours)	✓	✓	✓	✓	✓	S,G,R,I	Unified architecture with dynamic mode-switching

2.3. Research gap analysis

Based on our systematic analysis of current IDS approaches, three fundamental research gaps emerge that significantly limit the effectiveness of existing solutions in dynamic cybersecurity environments.

- **Limited Concept Drift Adaptation:** Current approaches typically excel at handling specific drift patterns but fail to provide comprehensive adaptation across the full spectrum of drift patterns. Analysis reveals that most IDS solutions focus primarily on sudden and gradual drift, with minimal support for incremental, recurring, or blip patterns. This pattern-specific adaptation bias creates significant vulnerabilities in dynamic environments where multiple drift types may occur simultaneously or sequentially, potentially allowing sophisticated attackers to exploit these adaptation blind spots.
- **Knowledge Preservation Deficits:** Most adaptive systems fail to preserve and transfer knowledge across different operational states or drift scenarios, forcing them to relearn patterns and relationships repeatedly. Few solutions effectively address this challenge by maintaining awareness of dormant features that may become relevant again or transferring learned relationships between features across different contexts. Without adequate knowledge preservation mechanisms, systems experience performance degradation during adaptation periods and require extensive retraining when encountering recurring attack patterns.
- **Integration Architecture Limitations:** Current frameworks struggle to integrate drift detection, adaptation mechanisms, and ensemble optimization into coherent architectures suitable for high-speed security environments. Existing approaches demonstrate fragmented implementations in which drift detection operates independently of adaptation strategies, resulting in suboptimal responses and increased computational overhead that limit the effectiveness of real-time deployment.

These research gaps collectively highlight the need for innovative IDS frameworks that can simultaneously adapt to multiple drift patterns, preserve knowledge across transitions, and efficiently integrate detection and adaptation mechanisms. Addressing these limitations requires novel architectural approaches that transcend current adaptation

frameworks while maintaining operational efficiency in dynamic cybersecurity environments.

Our proposed Incremental Genetic Programming Combiner with Mode-Switching (IGPC-MSOS) addresses these gaps by integrating multiple specialized learning modules (OSELM, FA-OSELM, and KP-OSELM) within a unified framework that leverages knowledge transfer to enhance both concept drift adaptation and feature evolution handling. By preserving and transferring knowledge across different operational modes and drift scenarios, our approach enables comprehensive adaptation to various forms of concept drift while effectively managing feature evolution.

Having identified critical research gaps in existing adaptive IDS approaches, we now present our IGPC-MSOS methodology, which explicitly addresses these limitations through a novel mode-switching architecture that integrates transfer learning and knowledge-preservation mechanisms.

3. Methodology

This section presents the IGPC-MSOS (Incremental Genetic Programming Combiner with Mode-Switching Online Sequential) methodology, which extends our previously developed IGPC framework [12] to address concept drift challenges in network intrusion detection. Building upon the original GPC approach [13], the proposed methodology introduces intelligent mode-switching capabilities to handle diverse drift patterns through dynamic operational mode selection.

The IGPC-MSOS methodology comprises six key components. First, the problem formulation mathematically defines the adaptive IDS challenge under concept drift conditions. Second, the method development presents a comprehensive approach for data handling and multi-classifier support. Third, the mathematical model of mode switching formalizes the intelligent switching mechanism between specialized operational modes. Fourth, the transition to mode-switching enhancement explains the evolution from the original IGPC framework. Fifth, the enhanced method development details the integration of Online Sequential Extreme Learning Machine (OSELM), Feature Adaptive OSELM (FA-OSELM), and Knowledge Preserving OSELM (KP-OSELM) modes. Finally, the algorithm development presents the implementation

framework with detailed algorithms for real-time adaptive intrusion detection.

The methodology addresses three critical limitations identified in the IGPC framework: single operational mode constraints during feature space evolution, insufficient capabilities for recurring pattern recognition, and performance stagnation during gradual concept drift. Through intelligent mode switching, IGPC-MSOS dynamically adapts its operational strategy based on detected drift characteristics and performance monitoring, providing comprehensive adaptability for evolving network security environments.

To illustrate the proposed framework, Fig. 1 presents the overall workflow of IGPC-MSOS. The process begins with incoming network traffic, followed by feature extraction and normalization. A drift detection and performance monitoring module then continuously evaluates the stream to determine whether sudden, gradual, incremental, or recurring concept drift is present. Based on these signals, the mode-switch controller dynamically selects the most suitable learner among OSELM, FA-OSELM, and KP-OSELM. The outputs of the selected learners are fed into the Genetic Programming (GP) combiner, which operates in three stages: (i) population initialization with candidate ensembles, (ii) fitness evaluation considering accuracy, stability, and computational cost, and (iii) evolutionary refinement through selection, crossover, and mutation. The best-performing ensemble then produces the final intrusion detection decision, ensuring adaptability and knowledge preservation in evolving network environments.

3.1. Problem formulation

Building upon our previously developed IGPC framework [12], which enhanced the original GPC approach [13] through incremental learning mechanisms, this paper introduces IGPC-MSOS (Incremental Genetic Programming Combiner with Mode-Switching Online Sequential) to address the remaining limitations in handling diverse concept drift patterns.

At its core, the problem revolves around processing and analyzing a continuous stream of network data, represented as $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, where each $x_i \in \mathbb{R}^d$ is a d -dimensional feature vector of network traffic and $y_i \in \{0, 1\}$ is the corresponding binary label indicating normal (0) or attack (1) traffic. The fundamental goal is to create an IDS that not only detects intrusions accurately but also adapts to the ever-changing landscape of network behaviors and attack patterns.

A critical challenge in this context is the phenomenon of concept drift in data streams. Concept drift refers to changes in the underlying data distribution over time, represented mathematically as $\exists t_1, t_2 : F_{t_1}(X, Y) \neq F_{t_2}(X, Y)$, where $F_t(X, Y)$ is the joint distribution of features X and labels Y at time t . These changes can manifest in various forms, such as sudden shifts in attack patterns, gradual evolution of network behaviors, or cyclical changes in traffic characteristics. Concept drift poses a significant challenge to traditional static IDS models, as it can rapidly render previously learned patterns and decision boundaries obsolete, leading to decreased detection accuracy and increased false positives or negatives.

A key aspect of the IGPC approach is its ability to detect the occurrence of concept drift in the data stream. Importantly, while the approach is designed to identify when drift occurs, it does not attempt to classify the specific type of drift (e.g., sudden, gradual, or recurring). This design choice focuses the approach on rapid adaptation to changes rather than expending computational resources on categorizing the nature of the drift. The drift detection can be formulated as a function $D(t)$ that signals the presence of drift at time t :

$$D(t) = \begin{cases} 1 & \text{if drift is detected at time } t \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

This binary approach to drift detection enables the system to quickly trigger adaptation mechanisms without the overhead of classifying drift types.

To address the challenge of concept drift, the IGPC approach must support multiple classifier types $\{Type_1, Type_2, \dots, Type_N\}$ and dynamically select and integrate them based on current data characteristics and drift detection. This multi-classifier approach enables the system to leverage diverse learning strategies, adapting its detection mechanisms to optimize performance in the current network environment. The approach must continuously monitor the incoming data stream for signs of drift, quickly identify when the underlying distribution has changed, and adapt its models accordingly, regardless of the specific nature of the drift.

The performance of the system at time t is quantified by a metric $P(t)$, which should be maximized over time: $\max_t P(t)$, subject to the constraint that $D_t \sim F_t(X, Y)$. This objective captures the need for the IDS to maintain high detection accuracy even as the nature of network traffic and attacks evolves. Simultaneously, the system aims to minimize the error $E(t)$ between ground truth GT and predicted values: $\min_t E(t) = \|GT - \text{Predicted}\|$. This error minimization is crucial for ensuring that the system's adaptations to concept drift result in tangible improvements in detection accuracy.

In essence, the IGPC approach aims to offer a comprehensive solution to the challenge of concept drift in network intrusion detection, with a focus on detecting and adapting to drift rather than merely categorizing it. The approach must operate in real-time, continuously processing incoming data, updating its models, and adjusting its strategies to maintain robust performance in the face of evolving network threats. This includes the ability to detect drift and respond with appropriate model updates or a retraining strategy, without the need to identify the specific type of drift. The approach should also be capable of leveraging historical data when beneficial, while giving more weight to recent observations that reflect the current state of the network.

This formulation encapsulates the core challenges of developing an adaptive IDS that remains effective and relevant in dynamic network environments. It sets the stage for developing innovative methods and algorithms to address the complex issue of concept drift, ensuring that the IDS maintains high detection rates and low false-positive rates as the characteristics of normal traffic and attack vectors change over time. The IGPC approach aims to push the boundaries of adaptive intrusion detection, creating a system that is not just reactive but predictive, capable of swiftly adapting to new types of network behaviors and potential security threats without the overhead of drift type classification.

Unlike existing approaches that address concept drift through static adaptation mechanisms or homogeneous ensemble methods, our formulation explicitly considers the multi-dimensional nature of concept drift in cybersecurity contexts. The problem requires simultaneous handling of feature relevance shifts, attack pattern evolution, and recurring threat behaviors—a challenge that cannot be adequately addressed through incremental enhancements to existing frameworks.

3.2. Rationale for selecting extreme learning machine (ELM)

Extreme Learning Machine (ELM) is a single-hidden-layer feedforward neural network (SLFN) where the hidden layer weights are randomly assigned and kept fixed, while the output weights are analytically determined using a least-squares solution. This design eliminates the need for iterative backpropagation and allows ELM to achieve extremely fast training and good generalization performance, even with large-scale datasets [53,54]. Unlike conventional neural networks, the randomization of hidden parameters and closed-form solution at the output layer provide a unique balance between efficiency and accuracy.

In the context of Intrusion Detection Systems (IDS), ELM is particularly effective due to several factors. First, IDS tasks often involve high-dimensional network traffic data and require models that can

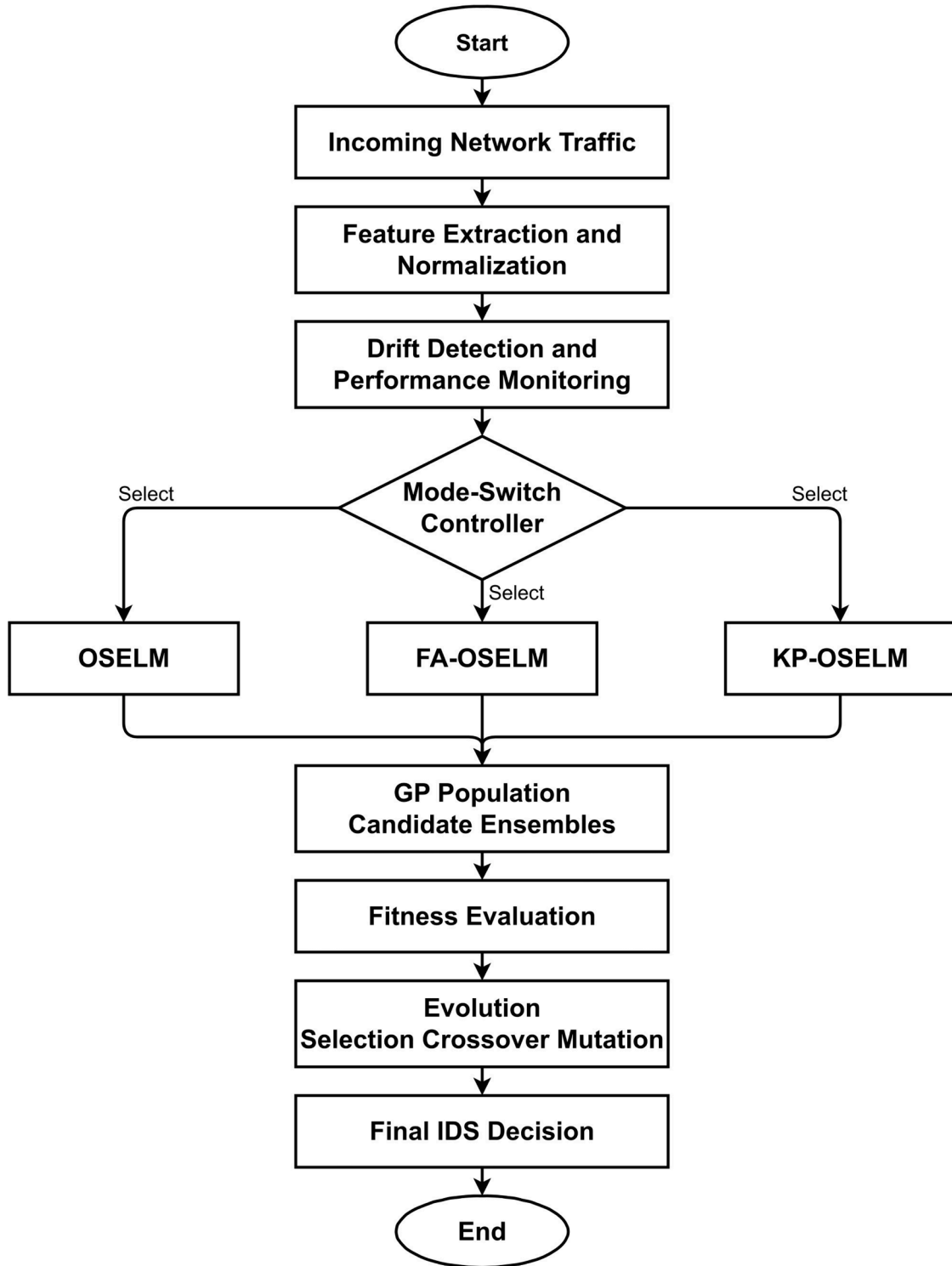


Fig. 1. Flowchart of the proposed IGPC-MSOS framework. The system adapts to concept drift through feature processing, drift detection, mode-switching among OSELM variants, and an evolutionary genetic programming combiner to produce the final intrusion detection decision.

update rapidly under streaming conditions. ELM's low computational complexity makes it suitable for real-time detection [55,56]. Second, IDS requires resilience to dynamic changes and evolving attack patterns; the analytical training of ELM helps prevent overfitting and maintains adaptability. Third, a growing body of research demonstrates the suitability of ELM variants for IDS, including on-line sequential ELM (OS-ELM) and feature-adaptive ELM, which ex-

tends its capabilities to handle concept drift and imbalanced datasets [57,58].

Therefore, we selected ELM as the basic classifier in the IGPC-MSOS framework because it provides (i) computational efficiency for large-scale, high-velocity network data, (ii) robustness in online adaptive learning scenarios, and (iii) a strong track record of success in IDS applications reported in the literature.

3.3. IGPC foundation framework

The IGPC framework [12] addresses the problem formulated through a comprehensive method that integrates several key components.

3.3.1. Data handling and preprocessing

The method begins with data normalization, crucial for handling features that span various orders of magnitude. The normalization process is defined by:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (2)$$

where x is an original feature value and x' is the normalized value. This scales all features to the range $[0, 1]$. Normalized data is stored in a buffer for chunk-based training and data balancing in subsequent stages.

3.3.2. Multi-classifier support

The approach supports multiple types of classifiers $\{Type_1, Type_2, \dots, Type_N\}$, allowing dynamic selection and integration based on the characteristics of the current data. The selection process can be represented as:

$$C_t = \arg \max_{Type_i} f(Type_i, D_t, \Delta_t) \quad (3)$$

Where C_t is the selected classifier at time t , f is a function that evaluates the suitability of the classifier, D_t is the current data chunk and Δ_t represents the detected drift.

3.3.3. Drift monitoring

Continuous monitoring of the data stream detects drift, which can be quantified as:

$$\Delta(D_t, D_{t-1}) > \theta$$

Where Δ measures the discrepancy between consecutive data chunks and θ is a threshold. When drift is detected, the approach initiates updates to the classifier.

3.3.4. Genetic programming selection rationale

Genetic programming was selected as the combiner mechanism after comparative analysis with alternatives, including neural architecture search (NAS) and reinforcement learning (RL). GP offers three critical advantages in the intrusion detection context: (1) interpretability of the resulting model, essential for security applications where understanding decision logic is required; (2) computational efficiency during adaptation - GP requires significantly fewer iterations to converge after drift compared to RL approaches; and (3) robust operation with limited training samples - GP can effectively operate with the smaller data chunks available during concept drift periods.

3.3.5. Prediction

The prediction process in the IGPC approach represents the culmination of its adaptive and ensemble-based approach to intrusion detection. This process dynamically adjusts based on the current state of the system, particularly in response to detected concept drift and the evolved ensemble structure.

When a new data point x arrives for classification, the approach first checks for the presence of concept drift. Let $D(t)$ be the drift detection function at time t :

$$D(t) = \begin{cases} 1 & \text{if drift is detected at time } t \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

The prediction process then proceeds as follows:

$$y_{pred} = \begin{cases} E_{current}(x) & \text{if } D(t) = 0 \\ E_{updated}(x) & \text{if } D(t) = 1 \end{cases} \quad (5)$$

Here, $E_{current}$ and $E_{updated}$ represent the current and updated ensemble models, respectively. It's important to note that these ensemble models are the result of the genetic algorithm-based optimization process described earlier.

In the case where no drift is detected ($D(t) = 0$), the current ensemble model $E_{current}$ is used. This model is the result of the most recent optimization cycle and represents the best-known aggregation of classifiers for the current data distribution. The prediction is computed as:

$$E_{current}(x) = A_{current}(\{C_i(x) | C_i \in S_{current}\}) \quad (6)$$

where $A_{current}$ is the current optimal aggregation function and $S_{current}$ is the current subset of classifiers.

When drift is detected ($D(t) = 1$), the approach triggers an update process. This involves re-optimizing the ensemble structure to adapt to the new data distribution. The updated ensemble $E_{updated}$ is then used for prediction:

$$E_{updated}(x) = A_{updated}(\{C_i(x) | C_i \in S_{updated}\}) \quad (7)$$

Here, $A_{updated}$ and $S_{updated}$ represent the newly optimized aggregation function and classifier subset, respectively.

The update process may involve several steps:

1. Retraining or fine-tuning individual classifiers with recent data.
2. Evolving a new ensemble structure through the genetic algorithm.
3. Adjusting the weights or parameters of the aggregation function.

This adaptive prediction process ensures that the IGPC approach remains responsive to changes in the underlying data distribution. By seamlessly switching between the current and updated models based on drift detection, the approach can maintain high prediction accuracy even in the face of evolving network behaviors and attack patterns.

Moreover, the use of ensemble methods in prediction provides several advantages:

1. Improved generalization: By combining multiple classifiers, the ensemble can often achieve better performance than any single classifier.
2. Robustness: The ensemble approach is less likely to be severely impacted by the failure or poor performance of a single classifier.
3. Flexibility: The genetic algorithm-optimised ensemble structure can capture complex relationships between classifiers, potentially uncovering synergies that improve overall prediction accuracy.

In summary, the prediction process in the IGPC approach is a dynamic and adaptive mechanism that leverages drift detection and ensemble learning to maintain high accuracy in the challenging domain of network intrusion detection. Its ability to swiftly adapt to changing conditions while utilizing the power of optimized classifier combinations makes it well-suited for dealing with the ever-evolving landscape of network security threats.

3.3.5.1. Algorithm Development. The IGPC approach [12] is implemented through several key algorithms, each playing a crucial role in the adaptive intrusion detection process. These algorithms work in concert to provide a comprehensive approach to handling concept drift and maintaining high detection accuracy in dynamic network environments. The approach follows a structured workflow in which incoming data streams are first processed by a buffer system. The system continuously monitors for concept drift and triggers the appropriate response mechanisms when it is detected. The approach incorporates three distinct operational modes:

1. **OSELN Mode:** Utilizes a Single-Layer Feedforward Network (SLFN) with inputs matching the current number of features, suitable for standard incremental learning scenarios.
2. **FA-OSELN Mode:** Employs transfer learning techniques with an SLFN structure adapted to the current feature set, which is effective for handling feature evolution, specifically designed to handle feature space changes.

3. **KP-OSELM Mode:** Implements a memory-enhanced SLFN with inputs configured for the maximum number of features, particularly effective for preserving knowledge in scenarios with recurring patterns.

The approach includes several key components working in concert:

- **Data Balancing:** Ensures proper distribution of classes in the training data
- **GP-Combiner:** Optimizes the combination of classifier outputs
- **Model Replacement:** Maintains ensemble quality by replacing underperforming models. This differs from the later mode-switching added to the algorithm to handle concept drift more effectively. In other words, the classifiers update and model replacement enable less aggressive model updates, while the mode switching enables more aggressive model updates.
- **Buffer Management:** Controls the flow and storage of incoming data

This architecture allows the approach to dynamically adapt to changing network conditions while maintaining consistent detection performance across various types of concept drift. The subsequent sub-sections present the different steps of the algorithms.

3.3.5.2. Initialization. The Initialization algorithm, presented as [Algorithm 1](#), serves as the starting point of the IGPC approach, responsible for setting up the initial set of classifiers. This algorithm is crucial for establishing a diverse set of base classifiers, which is essential to the ensemble's effectiveness in handling complex, evolving patterns in network traffic data. [Algorithm 1](#) takes two key parameters: NumHiddenNeurons and ActivationFunc (line 1). These parameters provide flexibility in the architecture of Online Sequential Extreme Learning Machine (OSELM) classifiers, enabling the creation of a diverse set of base classifiers. Each classifier can capture different aspects of the input data, thereby contributing to the ensemble's robustness.

The use of OSELM classifiers in [Algorithm 1](#) is significant as they offer fast learning speed and good generalization performance, making them well-suited for the dynamic nature of network intrusion detection. By allowing for multiple OSELM configurations, the algorithm sets the stage for an ensemble approach that can leverage the strengths of different classifier architectures.

The initialization process creates an OSELM classifier for each combination of hidden neuron count and activation function specified in the input parameters (lines 3–5). This approach yields a diverse initial set of classifiers, providing a robust starting point for handling the complex, evolving patterns in network traffic data. The algorithm returns this set of initialized classifiers (line 6), ready for use in subsequent stages of the IGPC approach.

Algorithm 1 Initialization of IGPC.

```

1: procedure INITIALIZE(NumHiddenNeurons, ActivationFunc)
2:   classifiers ← empty list
3:   for  $i = 1$  to len(NumHiddenNeurons) do
4:     classifiers.Append(OSELM(NumHiddenNeurons[i],
       ActivationFunc[i]))
5:   end for
6:   return classifiers
7: end procedure

```

3.3.5.3. Classifier Update. The Classifier Update algorithm, detailed in [Algorithm 2](#), is a critical component of the IGPC approach, responsible for adapting the classifiers to new data and maintaining an optimal set of classifiers. This algorithm is key to the approach's ability to adapt to concept drift by continuously refining and updating its classifier ensemble.

[Algorithm 2](#) takes several inputs, including the current set of base classifiers, new classifiers, training and validation data, and parameters for controlling the update process (line 1). It begins by updating each new classifier using labeled samples (lines 2–6). This update process is flexible, using either the standard OSELM update for incremental learning or transfer learning (TL) techniques for FA-OSELM and KP-OSELM modes. This flexibility allows the approach to adapt to different types of concept drift.

After updating, [Algorithm 2](#) evaluates the performance of each classifier on both the training and validation data (lines 4–5). This step is crucial for assessing the effectiveness of the updated classifiers. The insertion time is also recorded (line 6) and can be used to maintain diversity in the ensemble.

The updated classifiers are then sorted by a specified key (line 7), such as their validation score, training score, or insertion time. This sorting allows [Algorithm 2](#) to identify the best-performing classifiers. The top L classifiers are selected (line 8), where L is a parameter that controls the number of new classifiers to be incorporated into the base set.

Finally, [Algorithm 2](#) inserts the selected new classifiers into the base classifier set (lines 9–11). This step ensures that the ensemble is continuously updated with the most effective classifiers, allowing it to adapt to changes in the data distribution. The algorithm returns the updated set of base classifiers (line 12), ready for use in subsequent detection tasks.

Algorithm 2 Classifier update in IGPC.

```

1: procedure UPDATECLASSIFIER(baseClassifiers, newClassifiers,
   Xtrain, XValid, L, Key, LabeledSamples)
2:   for each clf in newClassifiers do
3:     Update clf using LabeledSamples (OSELM or TL for FA-
       OSELM and KP-OSELM)
4:     clf.trainScore ← accuracy(clf.predict(Xtrain), ytrain)
5:     clf.validScore ← accuracy(clf.predict(XValid), yvalid)
6:     clf.insertionTime ← current time
7:   end for
8:   newClassifiers ← sort(newClassifiers, key=Key)
9:   newClassifiers ← newClassifiers[:L]
10:  for each clf in newClassifiers do
11:    baseClassifiers.insert(clf)
12:  end for
13:  return baseClassifiers
14: end procedure

```

3.3.5.4. Model Replacement. The Model Replacement algorithm, presented as [Algorithm 3](#), is designed to maintain the quality of the classifier ensemble by removing underperforming models. This process is crucial for maintaining the ensemble's effectiveness over time, ensuring it remains adaptive and continues to perform well in the face of evolving network traffic patterns and attack strategies.

[Algorithm 3](#) takes the current set of base classifiers and a replacement method as inputs (line 1). It begins by defining a strategy dictionary (line 2) that maps replacement methods to specific criteria for classifier evaluation. This flexible approach allows different replacement strategies: 'best' uses the validation score, 'random' uses the insertion time, and 'wheel' uses the validation score with a different selection mechanism.

Based on the specified method, [Algorithm 3](#) selects the appropriate key for sorting (line 3). The base classifiers are then sorted by this key (line 4), effectively ranking them according to the chosen criterion. The algorithm then calculates a replacement threshold (lines 5–6), typically set to one-third of the total number of classifiers, so that the top two-thirds of classifiers are retained.

Finally, [Algorithm 3](#) selects the top-performing classifiers up to the threshold (line 7) and returns this refined set of base classifiers (line 8). By regularly removing underperforming classifiers, the algorithm ensures that the ensemble remains adaptive and continues to perform well in the face of evolving network traffic patterns and attack strategies.

Algorithm 3 Model replacement in IGPC.

```

1: procedure REPLACEMODEL(baseClassifiers, method)
2:   strategy ← {'best': 'validScore', 'random': 'insertionTime',
   'wheel': 'validScore'}
3:   key ← strategy[method]
4:   baseClassifiers ← sort(baseClassifiers, key=key)
5:   length ← len(baseClassifiers)
6:   threshold ← length/3
7:   baseClassifiers ← baseClassifiers[0:threshold]
8:   return baseClassifiers
9: end procedure

```

3.3.5.5. Decision Making. The decision-making algorithm, detailed in [Algorithm 4](#), is the final step in the IGPC approach and generates predictions based on the ensemble of classifiers. This algorithm leverages the ensemble's power, combining the predictions of multiple classifiers to produce a more robust and accurate final prediction.

[Algorithm 4](#) takes the set of base classifiers and a sample to be classified as inputs (line 1). It begins by initializing an empty list to store the decision profile (line 2) and then obtains probability predictions for the sample from each classifier in the base set (lines 3–5). These probability predictions form a decision profile that represents the ensemble's collective "opinion".

After collecting predictions from all classifiers, [Algorithm 4](#) converts the decision profile into a 2D array (line 6), where each row represents a classifier, and each column represents a class. The final decision is then made by averaging the probabilities across all classifiers (line 7). This approach, known as soft voting, allows each classifier to contribute to the final decision based on its confidence in each class prediction.

Finally, [Algorithm 4](#) determines the predicted class as the class with the highest average probability (line 8) and returns both the decision profile and the predicted class (line 9). By using probability outputs rather than hard class predictions, the algorithm can capture nuanced information from each classifier, potentially leading to better overall performance in complex intrusion detection scenarios.

Algorithm 4 Decision making in IGPC.

```

1: procedure MADEDECISION(baseClassifiers, sample)
2:   decisionProfile ← empty list
3:   for each clf in baseClassifiers do
4:     decisionProfile.add (clf.predictProbability(sample))
5:   end for
6:   decisionProfile ← array (decisionProfile, shape=(numOfClassifier, numOfClass))
7:   decisionProfile ← argmax(decisionProfile.sum(axis=0)/ decisionProfile.shape[0])
8:   ypred ← class of max(decisionProfile)
9:   return decisionProfile, ypred
10: end procedure

```

these [Algorithms 1–4](#) implement the core functionality of the IGPC approach. The approach's novelty lies in its combination of genetic programming for ensemble optimization, incremental learning through OSELM classifiers, and multi-mode support (OSELM, FA-OSELM, KP-OSELM) for handling various types of concept drift. This comprehensive approach allows the IGPC approach to maintain high detection accuracy in the face of evolving network behaviors and attack patterns, making

it well-suited for the challenging task of adaptive intrusion detection in dynamic network environments.

3.4. Enhanced mode switching mathematical model

The original GP-Combiner approach [13] is effective in combining multiple classifiers; however, it exhibits several critical limitations in handling concept drift. Let $C = \{c_1, c_2, \dots, c_n\}$ represent the set of base classifiers in the original system. The GP-Combiner prediction function can be expressed as:

$$y_{pred} = f_{GP}(C(x)) \quad (8)$$

where f_{GP} represents the genetic programming-based combination function. This formulation faces three significant constraints:

1. Static Adaptation Mechanism: The original system's response to concept drift is limited to:

$$\Delta(D_t, D_{t-1}) > \theta \implies \text{Update}(C) \quad (9)$$

where θ is a fixed threshold, leading to uniform responses regardless of drift type.

2. Feature Space Limitation: The system struggles with feature evolution, constrained by:

$$\dim(F_t) = \dim(F_{t-1}) \quad (10)$$

where F_t represents the feature space at time t .

3. Knowledge Preservation Gap: Historical pattern recognition is limited by:

$$P(y_t|x_t) \approx P(y_{t-1}|x_{t-1}) \quad (11)$$

Assuming continuous distribution similarity. The core innovation of our approach lies in the dynamic mode-switching mechanism that responds to both concept drift and performance stagnation. Let $\mathcal{M} = M_{OSELM}, M_{FA}, M_{KP}$ represent the set of available modes. The system state at time t is defined as:

$$S_t = (M_t, P_t, F_t, D_t) \quad (12)$$

where $M_t \in \mathcal{M}$ is the active mode, P_t represents the performance metrics, F_t denotes the feature space, and D_t indicates drift presence. The critical performance monitoring function evaluates system effectiveness since the last mode switch at time t_s :

$$\Phi(t, t_s) = \frac{1}{t - t_s} \sum_{i=t_s}^t P_i \quad (13)$$

Performance improvement since the last switch is measured by the following:

$$\Delta\Phi(t, t_s) = \Phi(t, t_s) - \Phi(t_s, t_s - w) \quad (14)$$

where w is the observation window before the last switch. The mode switching trigger function $\sigma(S_t)$ is then defined as:

$$\sigma(S_t) = \begin{cases} 1, & \text{if } (D_t = 1) \text{ or} \\ & (\Delta\Phi(t, t_s) < \epsilon \text{ and } t - t_s > \Delta t_{\min}) \\ 0, & \text{otherwise} \end{cases} \quad (15)$$

where:

$D_t = 1$ indicates drift detection $\Delta\Phi(t, t_s) < \epsilon$ represents performance stagnation $\Delta t_{\min}$ is the minimum time required between switches ϵ is the improvement threshold

When triggered ($\sigma(S_t) = 1$), the next mode is selected as:

$$M_{t+1} = \begin{cases} M_{FA} & \text{if } D_t = 1 \text{ and feature space changed} \\ M_{KP} & \text{if } D_t = 1 \text{ and recurring pattern detected} \\ M_{next} & \text{if stagnation, where } M_{next} \in \mathcal{M} \setminus M_t \end{cases} \quad (16)$$

3.4.1. Convergence analysis of mode switching

The mode-switching mechanism requires formal convergence guarantees to ensure stability. We define $\mathcal{M}_t$ as the mode at time t , with the mode transition probability matrix $P = [p_{ij}]$ where $p_{ij} = P(\mathcal{M}_{t+1} = j | \mathcal{M}_t = i)$. For our system to converge:

$$\lim_{t \rightarrow \infty} E[L(\mathcal{M}_t, D_t)] \leq \epsilon \quad (17)$$

where $L(\mathcal{M}_t, D_t)$ is the loss on data distribution D_t using mode $\mathcal{M}_t$, and ϵ is a small constant.

To prevent oscillation, we enforce a minimum time interval $\Delta t_{\min}$ between mode switches. This guarantees stability by ensuring:

$$P(|\mathcal{M}_{t+k} - \mathcal{M}_t| > 0) \leq \alpha e^{-\beta k} \quad \forall k > \Delta t_{\min} \quad (18)$$

where α and β are positive constants controlling the rate of decay in switch probability over time.

3.4.2. Catastrophic forgetting analysis for KP-OSELM

The KP-OSELM mode implements a memory mechanism that mitigates catastrophic forgetting. The forgetting rate γ for a pattern p after n updates is bounded by:

$$\gamma(p, n) \leq C \cdot \left(1 - \frac{|\mathcal{M}_p \cap \mathcal{H}|}{|\mathcal{H}|}\right)^n \quad (19)$$

where $\mathcal{M}_p$ represents the memory units activated by pattern p , $\mathcal{H}$ is the set of hidden neurons, and C is a normalization constant.

3.4.3. Distinction from existing adaptive ensembles

While existing adaptive ensembles like ADWIN-based approaches focus primarily on model weight adjustment or ensemble pruning, our mode-switching mechanism fundamentally differs in three aspects:

- **Operational Mode Transformation:** Rather than merely adjusting weights between fixed models, IGPC-MSOS transforms its entire operational architecture based on detected drift characteristics, enabling comprehensive adaptation beyond what weight-adjustment strategies can achieve.
- **Dual-Trigger Mechanism:** Unlike traditional ensembles that rely solely on explicit drift detection, our approach implements a dual-trigger system that responds to both explicit drift signals and performance stagnation, providing adaptation capability even when drift occurs below detection thresholds.
- **Cross-Mode Knowledge Transfer:** The mathematical formulation in Eq. (10) enables systematic knowledge transfer between operational modes, preserving critical detection patterns across adaptation cycles—a capability absent in conventional ensemble pruning strategies.

As illustrated in Fig. 2, the mode-switching mechanism integrates drift detection with performance monitoring to enable dynamic adaptation to evolving network conditions.

The mathematical formulation presented above establishes the theoretical foundation for mode-switching in adaptive intrusion detection systems. This formal representation highlights the advantages of dynamic mode selection over static operational modes, particularly in environments with multiple concurrent drift patterns. To understand how this theoretical framework translates into practical implementation, we next examine the transition from the original IGPC framework to the enhanced IGPC-MSOS approach, focusing on the specific enhancements that enable comprehensive adaptation to concept drift.

3.5. Enhancement justification and transition

The Incremental Genetic Programming Combiner (IGPC) approach [12], as described above, provides a robust foundation for adaptive intrusion detection in dynamic network environments. Its combination of genetic programming for ensemble optimization, incremental learning

through OSELM classifiers, and multi-mode support has demonstrated effectiveness in handling various types of concept drift. However, as network threats continue to evolve and become more sophisticated, there is a need for even greater adaptability and responsiveness in intrusion detection systems. To address this challenge, we propose an enhancement to the IGPC approach through the introduction of a mode-switching mechanism, as illustrated in Fig. 3.

Our previously developed IGPC framework [12] represented a significant advancement in adaptive intrusion detection through its innovative integration of genetic programming for ensemble optimization and incremental learning. The framework introduced three specialized operational variants: GPC-OS, utilizing a basic Online Sequential Extreme Learning Machine for standard incremental learning; GPC-FOS, employing Feature Adaptive OSELM for handling changes in the feature space; and GPC-KOS, implementing Knowledge Preserving OSELM for recurring pattern recognition. While each variant demonstrated superior performance in its respective drift scenario, the framework exhibited critical limitations in operational environments where multiple drift types coexist or rapidly transition between one another. Specifically, the framework lacked an integrated mechanism to dynamically switch between operational modes during runtime, requiring manual intervention when drift characteristics changed. Additionally, knowledge preservation remained confined within individual model boundaries, preventing effective cross-drift knowledge transfer, which is essential for complex security environments.

Furthermore, performance degradation was observed during transitions between different drift patterns, as each variant operated in isolation, without sharing knowledge. These operational constraints in environments with overlapping drift patterns provided the fundamental motivation for developing IGPC-MSOS, which extends the original framework by introducing intelligent mode-switching capabilities that dynamically select the optimal operational classifier based on both explicit drift detection and continuous performance monitoring.

The mode-switching capability, depicted in Fig. 3, builds upon the existing strengths of the IGPC approach while introducing a new level of dynamic adaptation. The approach begins with an initialization phase that establishes the GP-Combiner and initial classifiers. The data handling component processes incoming stream data via a buffer system and implements data-balancing techniques to ensure proper training distribution. The system then employs a dynamic mode selection mechanism that can switch between three operational modes as shown in Table 2:

A key innovation in this enhanced approach is the continuous performance monitoring system that triggers mode switches when performance stagnates over a defined period of time (dT). As shown in the figure, if the performance has not improved over the specified time window, the system automatically selects a different mode of operation, ensuring adaptive response to changing network conditions.

Where the original IGPC approach [12] could adapt to concept drift through incremental learning and ensemble optimization, mode switching enhancement allows the system to fundamentally alter its learning approach based on the detected characteristics of the current network environment. This addition is motivated by the observation that different types of concept drift and attack patterns may be better handled by different learning strategies.

In the following sections, we will detail the design and implementation of this mode-switching enhancement. We will explore how it integrates with the existing IGPC approach, the criteria for mode selection, and the mechanisms for smooth transitions between modes. This enhancement represents a significant step forward in adaptive intrusion detection, potentially improving the approach's ability to maintain high detection accuracy across a wider range of network conditions and attack scenarios.

The challenge of developing an adaptive Intrusion Detection System (IDS) in the face of evolving network behaviors and attack patterns

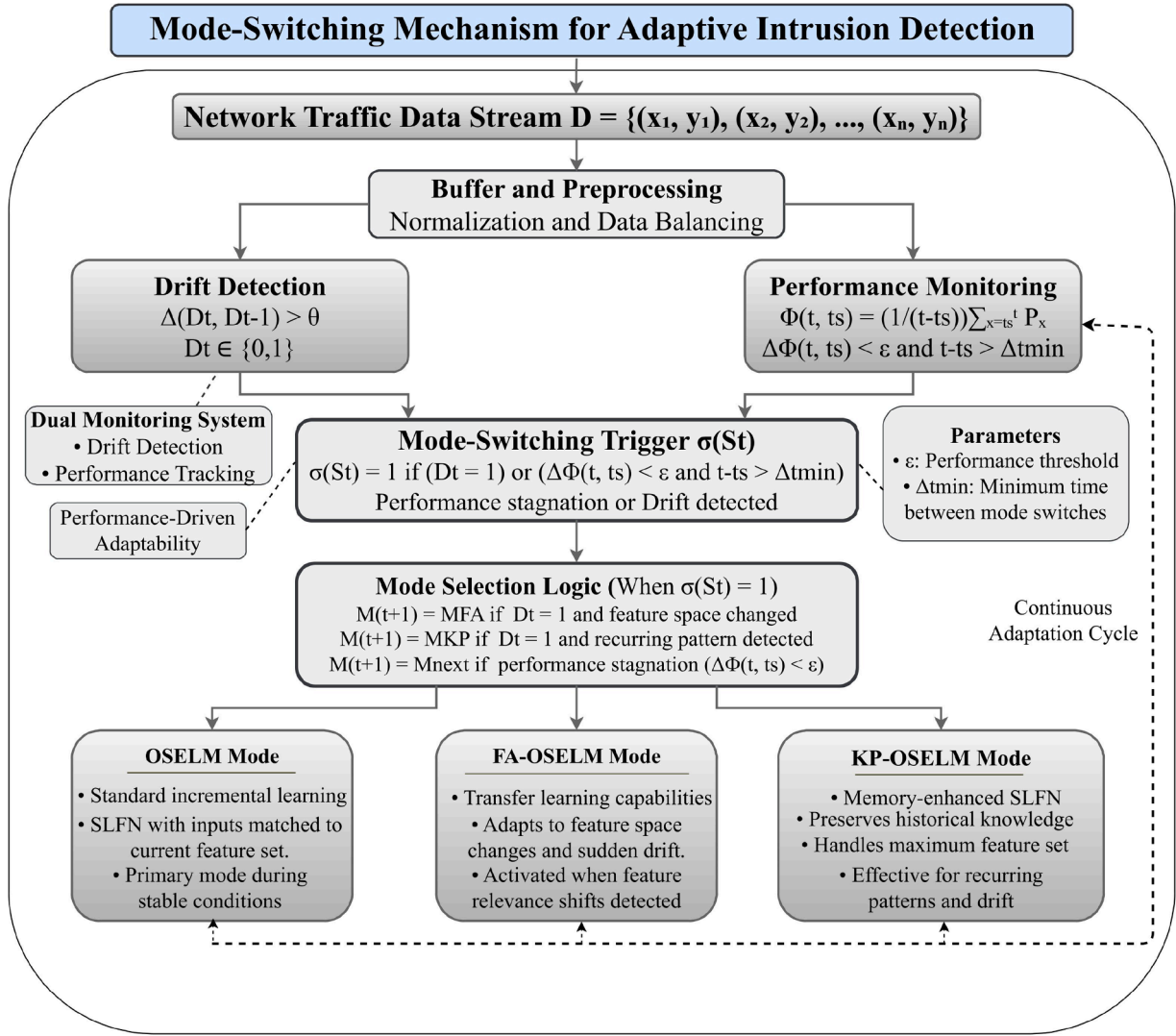


Fig. 2. Mode-switching mechanism for adaptive intrusion detection showing the integration of drift detection triggers (left), operational mode selection logic (center), and knowledge transfer pathways (right) that enable dynamic adaptation to evolving attack patterns.

Table 2
Comparison of operational modes in IGPC-MSOS.

Characteristic	OSELM Mode	FA-OSELM Mode	KP-OSELM Mode
Architecture	Standard SLFN with inputs matching current feature count	Transfer-enhanced SLFN with feature mapping capabilities	Memory-augmented SLFN with maximum feature capacity
Learning Mechanism	Incremental sequential learning	Transfer learning with domain adaptation	Knowledge preservation using memory cells
Primary Application	Standard incremental learning scenarios	Feature space evolution and sudden drift	Recurring patterns and cyclic drift
Activation Trigger	Default operational mode	Feature space changes detected	Recurring pattern recognition
Knowledge Management	Current data focus	Cross-domain knowledge transfer	Historical pattern preservation
Adaptation Strategy	Continuous incremental updates	Transfer-guided adaptation	Memory-enhanced recognition

can be formulated as follows: Given a continuous stream of network data.

$D = (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, where each $x_i \in \mathbb{R}^d$ represents a vector of dimensional features d of network traffic and $y_i \in \{0, 1\}$ is the corresponding binary label (0 for normal, 1 for attack), our aim is to develop an adaptive IDS that can effectively detect and dynamically respond to various types of concept drift. The system must maintain high detection accuracy over time, represented by a performance metric $P(t)$ at time t , while adapting to changes in the underlying data distribution. To achieve this, we define a drift function $\Delta(D_t, D_{t-1})$ that measures the

discrepancy between consecutive data chunks. The system's goal is to minimize the impact of drift on performance, such that:

$$\forall t : \Delta(D_t, D_{t-1}) > \delta \implies |P(t) - P(t-1)| < \epsilon \quad (20)$$

Where δ is a threshold for significant drift and ϵ is a small constant representing the maximum allowable performance degradation. This formulation ensures that the system responds dynamically to drift, maintaining consistent performance even as the underlying data distribution changes. The system must be capable of dynamically switching between different classifier modes $M = M_{OSELM}, M_{FA-OSELM}, M_{KP-OSELM}$,

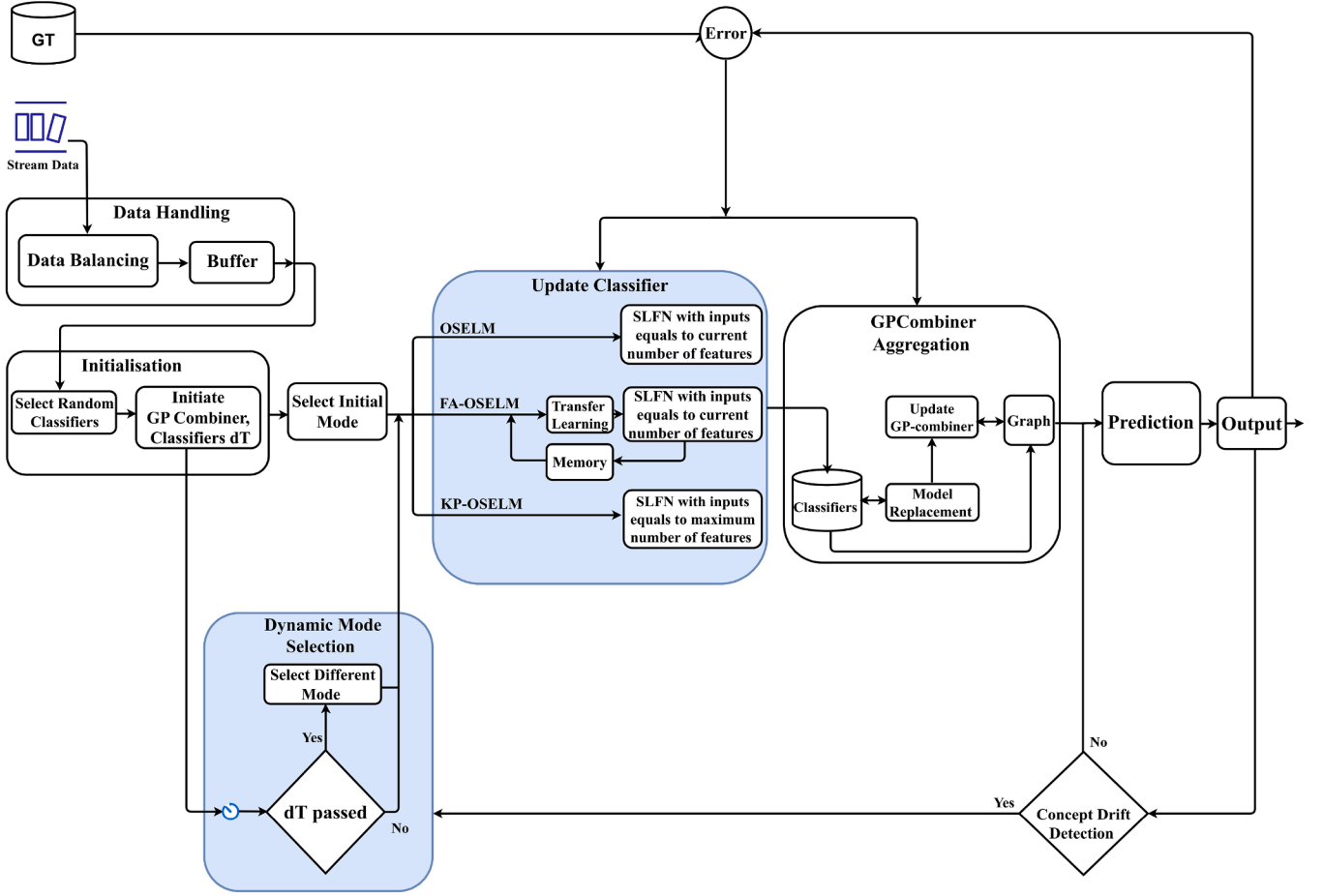


Fig. 3. IGPC-MSOS framework with dynamic mode-switching capability showing initialization, data handling, mode selection, and prediction processes.

each designed to handle specific types of drift. The mode selection at time t can be represented as:

$$m_t = \arg \max_{m \in M} f(m, D_t, \Delta(D_t, D_{t-1})) \quad (21)$$

Where f is a function that evaluates the suitability of each mode given the current data chunk D_t and detected drift $\Delta(D_t, D_{t-1})$. This formulation encapsulates the core challenges of real-time data handling, drift detection, adaptive model selection, and dynamic learning rate adjustment in a dynamic network environment. The goal is to create an IDS that not only detects intrusions accurately but also evolves its detection strategies in response to changing attack patterns and network behaviors, ensuring long-term effectiveness in the face of concept drift.

3.6. IGPC-MSOS enhanced architecture

To address the problem formulated above, we propose a comprehensive method that integrates several key components into a cohesive adaptive IDS approach. The foundation of our strategy, as illustrated in Fig. 3, lies in the continuous processing and analysis of incoming network data streams. As each new data chunk D_t arrives, it is pre-processed to ensure consistent feature scales and address class imbalance issues. This is achieved through a normalization function $N : \mathbb{R}^d \rightarrow [0, 1]^d$ and a balancing function $B : D \rightarrow D'$, where D' represents a balanced data set.

The preprocessed data are then passed through a drift detection mechanism $\Delta(D_t, D_{t-1})$ that continuously monitors changes in the underlying data distribution. This mechanism can identify various types of drift, including sudden, gradual, incremental, and recurring patterns. Upon detection of drift, the system triggers a mode selection

process to choose the most appropriate classifier from the set $M = \{M_{OSELM}, M_{FA-OSELM}, M_{KP-OSELM}\}$.

The mode selection process evaluates the nature of the detected drift and selects the most suitable classifier mode. The Online Sequential Extreme Learning Machine (OSELM) mode is employed for incremental learning on current feature sets, the Feature Adaptive OSELM (FA-OSELM) is utilized for transfer learning to adapt to feature space changes, and the Knowledge Preserving OSELM (KP-OSELM) is used for maintaining historical knowledge in recurring drift scenarios.

Once the appropriate mode is selected, the classifier is updated using the corresponding update mechanism. This adaptive update process ensures that the classifier remains relevant and effective in the face of evolving network patterns. The updated classifier then generates predictions for the current data chunk, which are further refined using a Genetic Programming (GP) Combiner. The GP Combiner, represented by a function $G : \mathbb{R}^k \rightarrow \mathbb{R}$, where k is the number of base classifiers, integrates outputs from multiple classifier instances to improve overall prediction accuracy through Bayesian inference.

The system's performance is continuously evaluated using metrics such as accuracy, F1-score, and recall. This evaluation process, denoted by $E(y_{pred}, y_{true})$, compares the predicted labels y_{pred} with the true labels y_{true} . If performance falls below a predefined threshold, the system initiates a mode-replacement process, selecting a higher-performing model from a maintained pool of alternatives.

Furthermore, the approach incorporates an adaptive mode-switching mechanism that monitors overall system performance across predefined intervals. This mechanism, represented by a function

$S(P_t, P_{t-1}, \dots, P_{t-k})$ evaluates the performance trend over the last k time steps and initiates mode switches if performance declines or

stagnates, ensuring the system remains responsive to changing network conditions.

3.7. Algorithm design and implementation

To improve clarity and consistency, the overall workflow of the proposed IGPC-MSOS framework is summarized in a single unified pseudocode (Algorithm 5). This end-to-end representation integrates the four fundamental building blocks: (i) initialization of base classifiers (Algorithm 1), (ii) classifier update procedure (Algorithm 2), (iii) model replacement strategy (Algorithm 3), and (iv) ensemble-based decision making (Algorithm 4).

In this unified view, the IDS begins by initializing a pool of classifiers using OSELM with specified hidden neurons and activation functions. Incoming network traffic is processed in sequential chunks, each undergoing feature extraction and buffering. A drift detector monitors the stream and triggers a mode-switching event when concept drift is detected, selecting the appropriate learner from OSELM, FA-OSELM, or KP-OSELM. Updated classifiers are then incorporated into the ensemble, where outdated or underperforming models are replaced. Finally, the genetic programming combiner aggregates predictions from the ensemble and produces the IDS decision. The overall process is also illustrated in Fig. 1, which provides a visual flowchart complementing the procedural description of Algorithm 5. Together, these references give a complete view of how IGPC-MSOS adapts to evolving network environments while preserving knowledge.

A detailed analysis of the computational complexity of each component of Algorithm 5 is provided in Section 3.8. This analysis quantifies the time and space costs of preprocessing, drift detection, mode-switching, OSELM-based updates, and the genetic programming combiner. By presenting both the unified pseudocode (Algorithm 5) and the flowchart (Fig. 1) alongside the complexity analysis, we provide a comprehensive methodological view that connects conceptual design, procedural execution, and computational feasibility of the proposed IGPC-MSOS framework.

Algorithm 5, therefore, acts as the umbrella procedure that orchestrates the complete adaptive learning process, while still maintaining explicit links to its component sub-procedures for detailed reference. It encapsulates the workflow of the IGPC-MSOS framework—beginning with data ingestion and preprocessing, through drift detection and mode selection, to classifier adaptation, replacement, and final ensemble decision-making. In each iteration, the system processes an incoming data chunk, updates the buffer, and invokes the drift detector to assess the presence and nature of concept drift. Upon detection, the framework dynamically switches among operational modes—OSELM, FA-OSELM, and KP-OSELM—depending on whether the drift corresponds to feature evolution, recurring patterns, or standard incremental adaptation. The initialization phase (cf. Algorithm 1) establishes a diverse set of base classifiers with heterogeneous hidden-layer configurations and activation functions, ensuring representational diversity within the ensemble. As new data arrive, these classifiers are refined via the update mechanism (cf. Algorithm 2), which selectively retrains candidates using recent labeled samples to preserve adaptability without overfitting. Concurrently, the model replacement module (cf. Algorithm 3) maintains ensemble quality by discarding weak classifiers and inserting more promising ones according to strategies such as best-score, random, or roulette-wheel selection. The final prediction stage leverages the Genetic Programming (GP) combiner (cf. Algorithm 4), which aggregates the outputs of the ensemble using evolutionary optimization to achieve an optimal trade-off between accuracy, stability, and computational efficiency. Through this hierarchical coordination, Algorithm 5 unifies individual procedures into a coherent, self-adaptive system capable of continuously learning from non-stationary data streams. It not only adapts to sudden or gradual concept drift but also preserves previously acquired knowledge, ensuring resilience to catastrophic forgetting. This design renders IGPC-MSOS a scalable and modular foundation for next-

generation intrusion detection systems operating under dynamic and uncertain network environments.

Algorithm 5 IGPC-MSOS framework with mode-switching mechanism.

```

1: procedure IGPC-MSOS(data_stream)
2:   Initialization: Initialize base classifiers using OSELM with given
    hidden neurons and activation functions (cf. Algorithm 1).
3:   Initialize genetic programming (GP) combiner, drift detector,
    and buffer.
4:   current_mode  $\leftarrow$  OSELM
5:   while data_stream not empty do
6:     chunk  $\leftarrow$  PreprocessData(data_stream.GetNextChunk())
7:     buffer.Add(chunk)
8:     if drift_detector.DetectDrift(chunk) then
9:       drift_type  $\leftarrow$  drift_detector.GetDriftType()
10:      new_mode  $\leftarrow$  SelectMode(drift_type)
11:      if new_mode  $\neq$  current_mode then
12:        current_mode  $\leftarrow$  new_mode
13:        SwitchClassifierMode(current_mode)
14:      end if
15:    end if
16:    Classifier Update: Update candidate classifiers with latest
    labeled samples and compute scores (cf. Algorithm 2).
17:    Model Replacement: Replace weak models based on strat-
    egy (best, random, roulette wheel) (cf. Algorithm 3).
18:    Decision Making: Aggregate ensemble predictions via GP
    combiner to produce the final IDS decision (cf. Algorithm 4).
19:    Output IDS decision; update GP population with fitness feed-
    back.
20:  end while
21: end procedure

```

3.8. Computational complexity analysis

The computational requirements of IGPC-MSOS can be characterized by examining the contributions of its principal components. Data handling and drift detection operate with a complexity of $O(nd)$, where n denotes the chunk size and d the feature dimensionality. The OSELM update stage incurs a cost of $O(nL^2 + L^3)$, with L representing the hidden layer size. When feature drift is detected, FA-OSELM introduces an additional transfer-learning overhead of $O(d_1 d_2 L + nL^2 + L^3)$, where d_1 and d_2 correspond to the original and updated feature dimensions. Similarly, KP-OSELM introduces memory-management operations amounting to $O(nL^2 + L^3 + ML)$, with M representing the number of retained memory samples.

The evolutionary search performed through Incremental Genetic Programming dominates the adaptation-phase complexity, scaling as $O(GPnC)$, where G is the number of generations, P the population size, and C the number of classifiers evaluated. Finally, the mode-switching mechanism itself is lightweight, requiring only $O(|\mathcal{M}|d)$ operations to evaluate all available modes and select the most appropriate one.

Overall, GP-based optimization constitutes the main computational bottleneck during adaptation cycles, whereas KP-OSELM dictates the primary space complexity through its memory component. Under the parameter settings used in our experiments, the total complexity amounts to approximately $O(10^6)$ operations per adaptation cycle, confirming that IGPC-MSOS remains computationally feasible for intrusion detection on dynamic network environments.

3.9. Stability and performance guarantees of mode-switching learning under concept drift

This section establishes the theoretical foundations of mode-switching learning under concept drift. We prove three main results:

(i) mode switching ensures stable learning under arbitrary drift patterns, (ii) performance degradation during adaptation is bounded, and (iii) noise affects only steady-state accuracy, not stability. All claims are stated with explicit assumptions, and the consequences of their violation are discussed.

3.9.1. Problem formulation and drift model

Consider a data stream $\{(x_t, y_t)\}_{t \geq 1}$ drawn from a time-varying distribution D_t . A learner maintains parameters $w_t \in \mathbb{R}^p$ and incurs instantaneous loss $\ell_t(w) = \ell(h(w; x_t), y_t)$. The population risk at time t is defined as

$$R_t(w) := \mathbb{E}_{(x,y) \sim D_t} [\ell(h(w; x), y)].$$

We model concept drift through the time-varying minimizer $w_t^* \in \arg \min_w R_t(w)$. The cumulative drift magnitude up to time T is quantified by the variation budget

$$V_T := \sum_{t=2}^T \|w_t^* - w_{t-1}^*\|, \quad (22)$$

where large individual increments correspond to abrupt drift and small persistent increments correspond to gradual drift.

3.9.2. Learning with mode switching

Let $\mathcal{F}_{t-1}$ denote the σ -algebra generated by all randomness up to time $t-1$, so that w_t is $\mathcal{F}_{t-1}$ -measurable. The stochastic gradient estimator $\tilde{g}_t(w_t)$, computed from a sample drawn independently from D_t , admits the decomposition

$$\tilde{g}_t(w_t) = \nabla R_t(w_t) + \varepsilon_t,$$

where $\nabla R_t(w_t)$ is the true gradient and ε_t is estimation noise satisfying $\mathbb{E}[\varepsilon_t | \mathcal{F}_{t-1}] = 0$ and $\mathbb{E}[\|\varepsilon_t\|^2 | \mathcal{F}_{t-1}] \leq \sigma^2$.

The learner alternates between two operational modes:

- **Tracking mode** ($m_t = 0$): Conservative updates for steady-state tracking.
- **Adaptation mode** ($m_t = 1$): Aggressive updates for rapid realignment after drift.

The corresponding update rule is

$$w_{t+1} = w_t - \eta_{m_t} \tilde{g}_t(w_t), \quad 0 < \eta_0 < \eta_1, \quad (23)$$

where η_0 and η_1 denote the step sizes for tracking and adaptation modes, respectively. The tracking error is defined as $e_t := w_t - w_t^*$.

3.9.3. Assumptions

Assumption 1 (Regularity of the risk). For all t , the risk $R_t(w)$ satisfies the Polyak–Łojasiewicz (PL) inequality:

$$\frac{1}{2} \|\nabla R_t(w)\|^2 \geq \mu (R_t(w) - R_t(w_t^*)).$$

The PL condition is implied by μ -strong convexity and L -smoothness but also holds for certain non-convex objectives, extending the applicability of our results. If the condition fails globally, the analysis remains valid locally around w_t^* or for convex last-layer training with frozen features, with stability holding within a basin of attraction.

Assumption 2 (Unbiased stochastic gradients). The gradient estimator satisfies

$$\mathbb{E}[\tilde{g}_t(w_t) | \mathcal{F}_{t-1}] = \nabla R_t(w_t), \quad \mathbb{E}[\|\tilde{g}_t(w_t) - \nabla R_t(w_t)\|^2 | \mathcal{F}_{t-1}] \leq \sigma^2.$$

If gradients are biased with $b_t := \mathbb{E}[\tilde{g}_t(w_t) | \mathcal{F}_{t-1}] - \nabla R_t(w_t) \neq 0$, stability persists provided $\sup_t \|b_t\|$ is bounded, with an additional bias term appearing in the steady-state error.

Assumption 3 (Step-size stability). The step sizes satisfy

$$0 < \eta_0 \leq \eta_1 < \frac{2\mu}{L^2}.$$

Table 3

System-wide parameters.

Parameter	Value	Description
seed	42	Reproducibility seed
device	cuda if available else cpu	Computation device

Table 4

Base ELM parameters.

Parameter	Value	Description
n_hidden	variable	Number of hidden neurons
activation	tanh	Activation function
C	1e5	Regularization constant
max_memory	10	Max memory bank size (KP-OSELM)
weight_scale	0.1	Initialization scale for weights

Table 5

Drift detection parameters.

Parameter	Value	Description
threshold	0.05	KS-test p-value threshold
epsilon	0.001	Stagnation epsilon
delta_t_min	10	Minimum stagnation window
warmup_chunks	5	Warm-up chunk count

This condition ensures the contraction factor $\rho(\eta) = 1 - 2\mu\eta + L^2\eta^2$ satisfies $\rho(\eta) < 1$ for both modes, guaranteeing geometric convergence toward w_t^* . If $\eta_1 \geq 2\mu/L^2$, contraction fails and the iterates may diverge regardless of the switching strategy. When L is unknown, conservative estimates via line search or setting $\eta_1 \approx \mu/L^2$ provide practical alternatives.

Assumption 4 (Drift detection interface). The mode-switching signal satisfies: (i) detection delay is bounded by Δ after any true drift, (ii) the number of false alarms up to time T is bounded by $N_{FA}(T)$, and (iii) each alarm triggers at most L adaptation steps.

Unbounded dwell time inflates variance bounds but does not compromise stability. Similarly, unbounded false alarms increase cumulative regret without destabilizing the system.

4. Experimental design

This section outlines the experimental design adopted for evaluating the IGPC-MSOS framework. The design integrates system-wide execution parameters, base learner configurations, drift-detection settings, core IGPC-MSOS hyperparameters, dataset-specific configurations, and the controlled feature drift schedule. These components collectively ensure reproducibility, controlled drift simulation, and consistent benchmarking across all intrusion detection datasets.

4.1. System-wide configuration

To guarantee reproducibility and consistent hardware behavior, the system-wide parameters shown in [Table 3](#) was fixed for all experiments.

4.2. Base ELM and variant configuration

All IGPC-MSOS modes (OSELM, FA-OSELM, KP-OSELM) rely on consistent base ELM configurations. These foundational hyperparameters are summarised in [Table 4](#).

4.3. Drift detection parameters

Concept drift activation is triggered using a KS-test p-value threshold, performance stagnation checks, and warm-up periods. [Table 5](#) lists the parameters governing drift detection.

Table 6
IGPC-MSOS core parameters.

Parameter	Value	Description
n_classifiers_per_mode	4	Classifiers per operational mode
perf_threshold	0.8	Minimum acceptable performance
drift_threshold	0.02	Drift reactivity threshold
gp_pop_size	200	GP population size
gp_ngen	10	Number of GP generations
initial_mode	OSELM	Initial operational mode

Table 7
Dataset-specific configurations.

Dataset	Label Column	Attack Label	Hidden Sizes	Chunk Size
KDD99	label	1	512	500
IDS ISCX 2012	tcp.time_delta	0	512	1000
CIC IDS 2017	Label	0	512	1000
CSEICIDS2018	Label	0	512	1000
HIKARI2021	Label	1	512	1000

4.4. IGPC-MSOS core hyperparameters

Global hyperparameters for classifier population, drift-sensitivity, and genetic programming optimization are presented in Table 6.

4.5. Dataset-specific configuration

We evaluated our approach on five benchmark datasets representing diverse network environments and attack scenarios:

- **KDD99** [59]: Classic intrusion detection dataset with 41 features, including multi-class attack categorization (Normal, DoS, Probe, R2L, U2R) suitable for comprehensive classification evaluation. This dataset contains 4,898,431 connection records derived from the 1998 DARPA Intrusion Detection Evaluation Program.
- **ISCX2012** [60]: Real-world network traffic dataset with various intrusion scenarios, providing challenging concept drift patterns and realistic network behavior simulation. Contains both normal activities and synthetic attack scenarios in a controlled testbed environment.
- **CICIDS2017** [61]: Modern dataset with diverse attack types and 78 features, incorporating contemporary attack vectors including DDoS, Port Scan, and Infiltration attacks. Generated using real network traffic and updated attack scenarios to reflect current cybersecurity challenges.
- **CSE-CIC-IDS2018** [61]: Updated dataset with modern attack patterns and 80 features, including advanced persistent threats and sophisticated attack methodologies. Incorporates IoT-based attacks and reflects contemporary network security challenges.
- **HIKARI2021** [62]: Recent dataset focusing on IoT network security with 85 features, representing emerging security challenges in Internet of Things environments. Specifically designed to address concept drift in IoT network traffic patterns.

Each dataset includes its own label column, attack label value, hidden-layer size, and chunk size. Table 7 summarises these dataset-specific experimental parameters.

4.6. Feature drift schedule

To evaluate the adaptability of IGPC-MSOS under controlled concept drift, a predefined feature-removal schedule was imposed. This simulates sudden, gradual, and recurring drift patterns. Table 8 lists all feature removal events. All datasets use only drifts up to chunk 21, except HIKARI2021, which uses the full schedule.

Together, these configuration components form a rigorous, reproducible, and comprehensive experimental environment for evaluating IGPC-MSOS under diverse and evolving network conditions.

Table 8
Feature changes schedule.

Chunk Index	Removed Features (Indices)
15	[0, 2]
20	[3]
21	[0, 2]
22	[0, 2]
23	[0, 2]
24	[0, 2]
30	[1, 4, 6]
31	[1, 4, 6]
60	[10, 12, 50]
61	[10, 12, 50]
62	[10, 12, 50]
63	[10, 12, 50]
64	[10, 12, 50]
65	[10, 12, 50]
121	[55]
122	[55]
123	[55]
144	[55]
145	[55]
172	[55]
173	[44, 45, 55, 76]
174	[44, 45, 55, 76]
175	[44, 45, 55, 76]
176	[44, 45, 55, 76]
177	[44, 45, 55, 76]

5. Results and analysis

This section presents a comprehensive empirical evaluation of the proposed IGPC-MSOS framework. Experiments assess predictive performance, temporal stability under concept drift, computational efficiency, and comparative robustness against established benchmark systems.

5.1. Performance evaluation metrics

The evaluation employs F1-score, accuracy, and inference latency as the primary indicators of IDS performance. Comparative experiments are conducted between our proposed method (IGPC-MSOS), the operational modes of the IGPC family (OS, FOS, KOS) [12], and two new state-of-the-art baselines: FlowTransformer [63] and the Adaptive Tree-like Neural Network (ATNN) [5]. The subsequent analysis is organized as follows: Section 5.2 presents an ablation study comparing IGPC-MSOS against its constituent single-mode variants to validate the mode-switching hypothesis. Section 5.3 examines the temporal dynamics of mode selection across datasets. Finally, Section 5.6 positions IGPC-MSOS against state-of-the-art baselines to establish broader competitiveness.

5.2. Ablation study: mode comparison across datasets

To validate the mode-switching hypothesis and measure the contribution of each operational mode, we conduct an ablation study comparing the three single-mode variants—GPC-OS (OSELM mode utilizing standard incremental learning), GPC-FOS (FA-OSELM mode employing transfer learning techniques for feature adaptation), and GPC-KOS (KP-OSELM mode implementing knowledge preservation mechanisms for recurring pattern detection)—against the full mode-switching architecture IGPC-MSOS. The goal is to evaluate whether dynamic mode selection provides measurable advantages over static learning regimes under different concept drift patterns. This analysis answers the fundamental question: *Does the added complexity of mode-switching deliver tangible performance improvements, or could a single well-tuned mode suffice?*

Figs. 4–8 present the F1-score trajectories for all five datasets, while Table 9 summarizes the superiority of IGPC-MSOS across datasets.

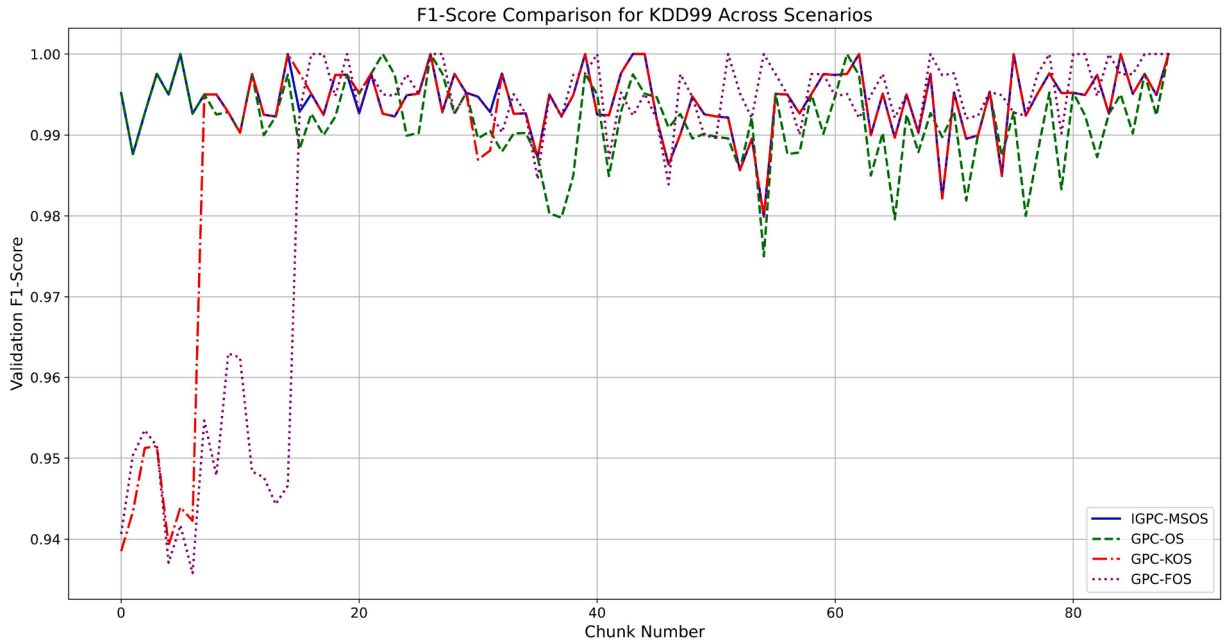


Fig. 4. F1-score comparison for the KDD99 dataset.

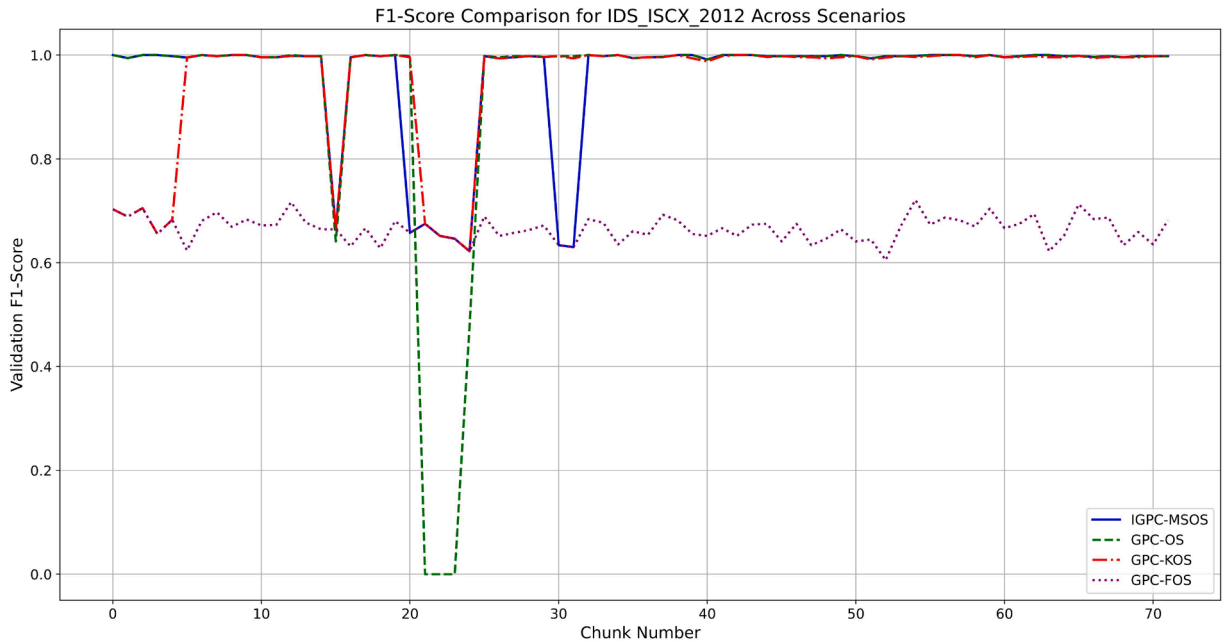


Fig. 5. F1-score comparison for the IDS-ISCX2012 dataset.

5.2.1. KDD99 dataset: mode comparison

As illustrated in Fig. 4, all four modes maintain high F1-scores across the KDD99 stream, reflecting the relatively mild drift in this dataset. However, the IGPC-MSOS curve consistently sits at the top of the performance band. Although the improvement is modest, the mode-switching architecture maintains a persistent lead over OS, KOS, and especially FOS. This confirms that IGPC-MSOS is able to exploit complementary strengths from different learners even when drift is limited. These findings are reflected in the KDD99 entry of Table 9, which categorizes IGPC-MSOS as “Slightly superior.”

5.2.2. IDS-ISCX2012 dataset: mode comparison

Fig. 5 highlights the importance of mode switching on the IDS-ISCX2012 dataset. FOS undergoes severe collapses, KOS experiences

moderate instability, and OS shows noticeable performance fluctuations under mixed real and virtual drift. IGPC-MSOS, however, maintains a consistently high F1-score and demonstrates substantially greater stability. This dataset is where the advantage of dynamic mode selection is most pronounced. Accordingly, Table 9 labels IGPC-MSOS as “Strongly superior,” reflecting the significant improvements observed.

5.2.3. CIC-IDS2017 dataset: mode comparison

Fig. 6 shows that the OS mode achieves the highest overall F1-score on CIC-IDS2017. The IGPC-MSOS curve follows closely and remains clearly above those of KOS and FOS. In this case, a single learner OS is particularly well-aligned to the dataset’s drift structure. IGPC-MSOS therefore performs slightly below OS, as noted in Table 9, which

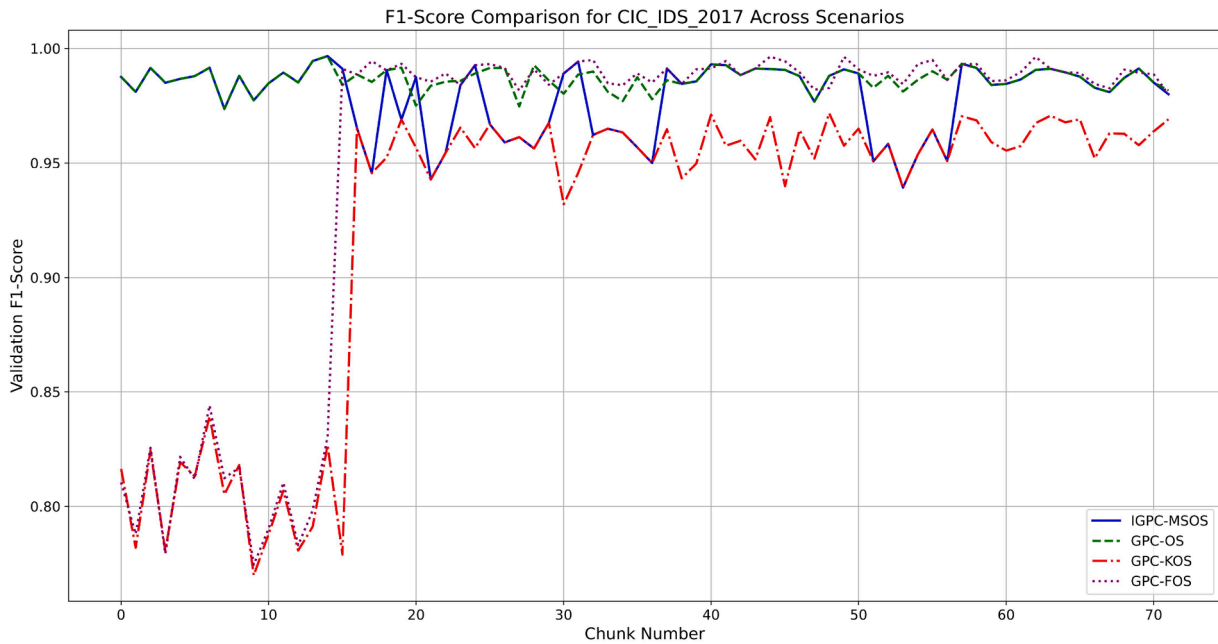


Fig. 6. F1-score comparison for the IDS2017 dataset.

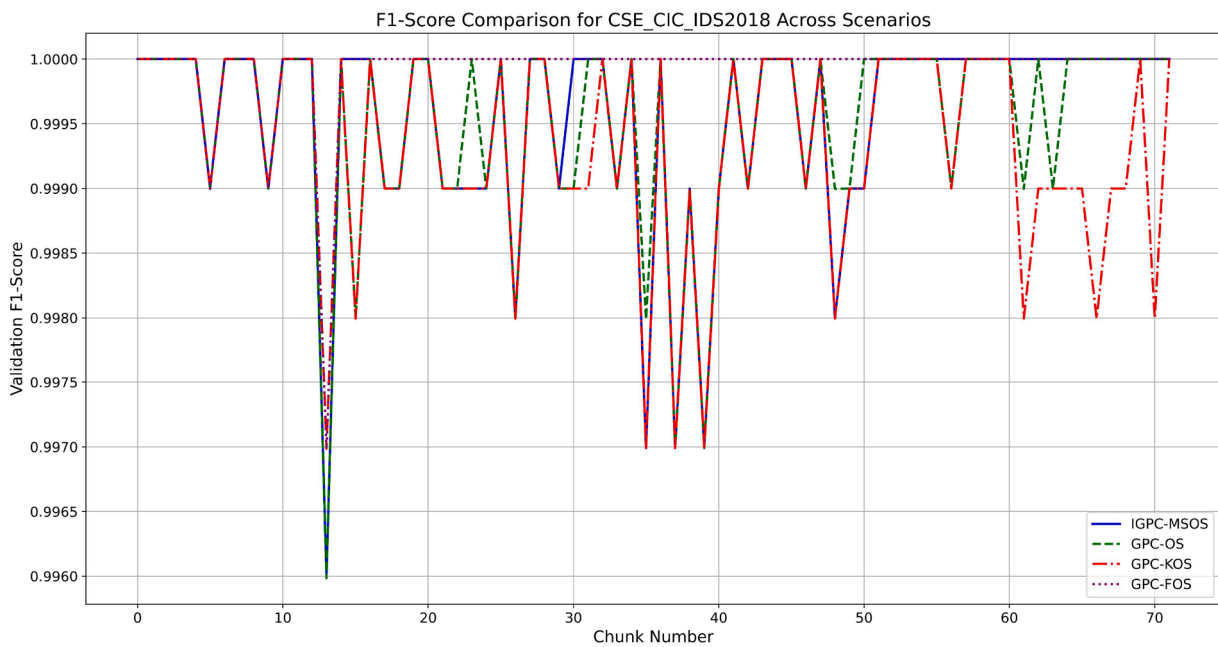


Fig. 7. F1-score comparison for the CSE-CIC-IDS2018 dataset.

categorizes this dataset as “Slightly inferior to OS.” Nevertheless, IGPC-MSOS avoids the dramatic drops seen in FOS or KOS, maintaining robust performance across all chunks.

5.2.4. CSE-CIC-IDS2018 dataset: mode comparison

As shown in Fig. 7, all modes achieve near-perfect F1-scores across the entire stream for CSE-CIC-IDS2018, with curves clustered tightly above 0.997. IGPC-MSOS performs on par with OS and FOS, demonstrating that mode switching does not compromise accuracy when all learners naturally handle the dataset well. This is reflected in Table 9, where IGPC-MSOS is classified as “Equivalent to best mode” for this dataset.

5.2.5. HIKARI-2021 dataset: mode comparison

Fig. 8 illustrates that all modes perform nearly identically across the long HIKARI2021 stream, despite multiple drift events. IGPC-MSOS closely overlaps with OS, FOS, and KOS throughout the entire evaluation, demonstrating that the switching mechanism does not introduce instability or unnecessary mode transitions. This behavior is reflected in Table 9, where IGPC-MSOS is listed as “Equivalent to best mode.”

Summary of mode-switching superiority

The results across Figs. 4–8, together with the summary in Table 9 reveals several distinct behavioral regimes that clarify when and why the mode-switching strategy provides an advantage. On IDS-ISCX2012 (Fig. 5), where concept drift is severe and highly unstable, IGPC-MSOS

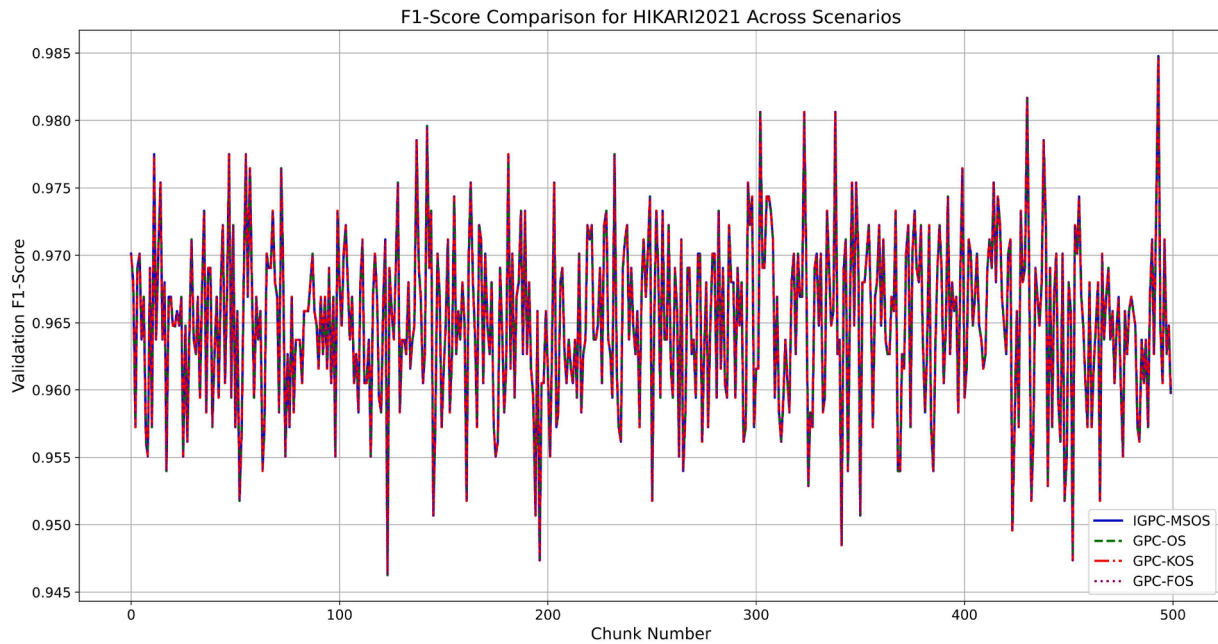


Fig. 8. F1-score comparison for the HIKARI2021 dataset.

Table 9
Summary of IGPC-MSOS superiority across datasets.

Dataset	IGPC-MSOS Superiority	Interpretation
KDD99	Slightly superior	Consistent marginal gains on mild drift
IDS-ISCX2012	Strongly superior	Clear improvements under severe mixed drift
CIC-IDS2017	Slightly inferior to GPC-OS	OS best suited; IGPC-MSOS remains robust
CSE-CIC-IDS2018	Equivalent to best mode	All modes saturate near-perfect accuracy
HIKARI2021	Equivalent to best mode	Long-stream stability with no degradation

demonstrates clear superiority over all single-mode learners, dynamically adapting as drift conditions evolve and maintaining stable, high-level performance where individual modes fail. In contrast, KDD99 (Fig. 4) exhibits only mild drift; here the benefits of switching are modest but consistent, reflecting the system's ability to exploit subtle complementarities across modes without introducing variance.

A different pattern appears on HIKARI2021 and CSE-CIC-IDS2018 (Figs. 8 and 7), where all learners achieve tightly clustered, near-saturated performance. In these stable environments, IGPC-MSOS matches the best-performing single mode, showing that the switching mechanism introduces no unnecessary overhead when drift is limited or easily managed. Conversely, CIC-IDS2017 (Fig. 6) represents a case in which a single mode—specifically OS—naturally aligns with the dataset's incremental drift structure. Here, IGPC-MSOS performs slightly below the optimal single learner, though it still surpasses FOS and KOS and maintains robust performance across all chunks.

These observations reveal distinct behavioral regimes where mode-switching provides varying degrees of advantage. The evidence indicates that dynamic mode selection is most beneficial under complex or unstable drift conditions (IDS-ISCX2012), remains neutral under stable regimes (HIKARI2021, CSE-CIC-IDS2018), and concedes marginally when a particular mode naturally aligns with the drift characteristics (CIC-IDS2017, where OS dominates). We now examine the quantitative performance metrics to validate these qualitative observations.

5.2.6. Quantitative comparison of modes across datasets

To complement the qualitative interpretation of Figs. 4–8, Table 10 reports the numerical performance of all operational modes—IGPC-MSOS, GPC-OS, GPC-KOS, and GPC-FOS—across the five benchmark datasets. Metrics include validation F1, accuracy, and inference time,

Table 10
Performance of IGPC-MSOS and single-mode variants across datasets.

Dataset	Mode	F1 Score	Accuracy	Inference Time (s)
KDD99	IGPC-MSOS (Ours)	0.994083	0.990472	0.001464
	GPC-OS	0.983622	0.985472	0.001625
	GPC-KOS	0.981084	0.980051	0.001432
	GPC-FOS	0.980331	0.978831	0.001478
IDSISCX2012	IGPC-MSOS (Ours)	0.959091	0.940389	0.000817
	GPC-KOS	0.938760	0.929444	0.000756
	GPC-OS	0.931297	0.911861	0.000871
	GPC-FOS	0.666113	0.502194	0.000770
CICIDS2017	GPC-OS	0.981609	0.982917	0.001620
	IGPC-MSOS (Ours)	0.978868	0.979611	0.001568
	GPC-FOS	0.951184	0.934917	0.001519
	GPC-KOS	0.917938	0.901556	0.001471
HIKARI2021	IGPC-MSOS (Ours)	0.965128	0.932677	0.001675
	GPC-FOS	0.965128	0.932677	0.001643
	GPC-KOS	0.965128	0.932677	0.001637
	GPC-OS	0.961121	0.931265	0.001741
CSECICIDS2018	GPC-FOS	0.999911	0.999361	0.001758
	IGPC-MSOS (Ours)	0.999641	0.999093	0.001828
	GPC-OS	0.999113	0.999028	0.001999
	GPC-KOS	0.999018	0.998539	0.001902

enabling deeper examination of both performance and computational efficiency.

The numerical results in Table 10 provide a detailed quantitative view of how the different modes behave across the five datasets and reinforce the ablation findings. On KDD99, IGPC-MSOS attains the highest F1-score (0.9941). Across this dataset, all modes perform well, and

the mode-switching strategy consistently yields a measurable advantage while maintaining inference times comparable to those of the single-mode variants. A substantially different pattern emerges in IDS-ISCX2012, where the drift characteristics are more severe. Here, IGPC-MSOS records the top F1-score (0.9591), far surpassing GPC-FOS, which collapses to 0.6661 under mixed drift. This dataset highlights the importance of adaptive switching, and IGPC-MSOS also remains among the fastest modes with a latency of 0.000817 seconds.

In HIKARI2021, all modes achieve nearly identical F1-scores (0.9651), reflecting the shallow drift structure of the dataset. IGPC-MSOS performs on par with the best single mode while retaining low computational overhead, demonstrating that the switching mechanism introduces no unnecessary complexity when the data stream is stable. For CICIDS2017, GPC-OS emerges as the best-performing single mode, with an F1-score of 0.9816, consistent with the dataset's incremental drift. Although IGPC-MSOS trails slightly with an F1-score of 0.9789, it avoids the instability seen in FOS and KOS and remains robust across all data chunks. Finally, CSECICIDS2018 represents a saturation regime where all modes achieve extremely high performance (F1-scores above 0.9991). Under these conditions, differences between modes become negligible; IGPC-MSOS trails GPC-FOS only marginally, by less than 0.0004.

The quantitative analysis confirms that no single mode is universally optimal across all drift conditions. IGPC-MSOS consistently achieves the strongest results on drift-heavy datasets, performs equivalently to the best single mode when drift is minimal or benign, and is only slightly inferior when a particular mode aligns naturally with the dataset's drift pattern, as in CICIDS2017. Across all settings, inference times remain low and comparable to those of the single-mode baselines, confirming that the adaptive mode-switching strategy delivers real-time performance without introducing computational overhead that would compromise deployment feasibility.

Ablation study summary

The ablation analysis demonstrates that IGPC-MSOS achieves superior or equivalent performance to the best single mode in four of five datasets, with particularly strong advantages on drift-intensive datasets (IDS-ISCX2012). The quantitative results confirm that dynamic mode-switching provides measurable benefits without introducing computational overhead, validating the core architectural hypothesis. Having established the effectiveness of mode-switching through internal comparison, we next examine the temporal dynamics of mode selection to understand when and why the controller activates specific operational modes.

5.3. F1 score evolution over time (mode-switching mechanism only)

To examine temporal behavior under concept drift, we analyze the F1-score evolution *exclusively* for the proposed IGPC-MSOS mode-switching mechanism. The colored trajectories reveal not only the model's predictive performance over time but also the internal mode-switching decisions activated in response to drift events. For each dataset, these patterns illuminate how IGPC-MSOS detects, reacts to, and recovers from dynamic changes in the data distribution. Figs. 9–13 summarizes the time-series interactions between F1-score stability and operational mode selection.

5.3.1. KDD99 dataset: F1-score evolution

As shown in Fig. 9, IGPC-MSOS begins with the OSELM mode (blue), which handles the initially stable stream effectively. Shortly thereafter, the system transitions into KP-OS-ELM (red), which dominates the remainder of the evaluation. This transition aligns with the recurrent nature of KDD99, where attack patterns repeat periodically. The brief appearance of FA-OS-ELM (green) suggests minimal feature drift, insufficient to justify long-term activation. The consistently high F1-scores (mostly above 0.992) indicate that the controller identifies KP-OS-ELM

as the most appropriate long-term strategy, providing superior stability under mild drift.

5.3.2. CIC-IDS2017 dataset: F1-score evolution

In Fig. 10, OSELM (blue) dominates the mode selection process, with FA-OS-ELM (green) and KP-OS-ELM (red) appears only sporadically. This behavior indicates that CIC-IDS2017 contains drift patterns well suited for incremental learning rather than recurrent or feature-based drift. The controller quickly abandons FA-OS-ELM and KP-OS-ELM when they fail to enhance performance, reverting to OSELM as the preferred mode. This pattern explains why OSELM achieves the highest standalone performance on this dataset.

5.3.3. CSE-CIC-IDS2018 dataset: F1-score evolution

Fig. 11 demonstrates minimal mode switching across the CSE-CIC-IDS2018 stream. Nearly all F1 values exceed 0.997, and the controller predominantly selects KP-OS-ELM (red), with rare transitions into OS-ELM (blue) or FA-OS-ELM (green). This indicates that the dataset exhibits limited drift and memory-based adaptation suffices to preserve stability. The absence of rapid switches highlights the controller's ability to avoid unnecessary transitions when performance remains optimal.

5.3.4. IDS-ISCX2012 dataset: F1-score evolution

As shown in Fig. 12, the IDS-ISCX2012 dataset exhibits the most complex and volatile mode switching pattern. The system alternates frequently between OSELM (blue) and KP-OS-ELM (red), while FA-OS-ELM (green) appears only in brief intervals. This frequent switching reflects the dataset's severe mixed drift, consisting of real, virtual, and recurrent components. The rapid transitions highlight IGPC-MSOS's ability to adapt dynamically: OSELM manages immediate changes, while KP-OS-ELM stabilizes performance during recurring conditions. The highly variable F1 values reinforce the drift difficulty and demonstrate the importance of adaptive selection.

5.3.5. HIKARI-2021 dataset: F1-score evolution

Fig. 13 illustrates dense interweaving of all three modes across the extended 500-chunk stream. Unlike the previous datasets, no single mode dominates for long periods. This frequent switching indicates a dataset characterized by shallow, distributed micro-drift rather than consistent real or recurring drift. Importantly, the F1-score remains highly stable (0.945–0.985), confirming that rapid mode alternation does not introduce instability. Instead, IGPC-MSOS flexibly leverages small local advantages of different modes at different times.

Cross-dataset interpretation of mode-switching behavior

Across Figs. 9–13, the IGPC-MSOS controller exhibits three characteristic behavioral patterns that reflect its adaptive response to diverse drift conditions. In CIC-IDS2017 (Fig. 10), The system converges almost exclusively to the OSELM mode, indicating that this dataset's predominantly incremental drift structure is best handled by a single incremental learner. In contrast, KDD99 (Fig. 9) and CSE-CIC-IDS2018 (Fig. 11) demonstrate a stable preference for KP-OS-ELM, where the recurrent or low-intensity drift present in these datasets favors the memory-preserving capabilities of the kernel-based mode. A markedly different dynamic emerges in IDS-ISCX2012 (Fig. 12) and HIKARI2021 (Fig. 13), where the controller alternates rapidly between modes in response to highly unstable, mixed, or micro-drift environments. These observations are summarized in Table 11, which consolidates the dominant mode and its interpretation for each dataset. Together, these patterns confirm that IGPC-MSOS accurately characterizes the underlying drift behavior and selects operational modes that balance predictive performance, stability, and adaptability.

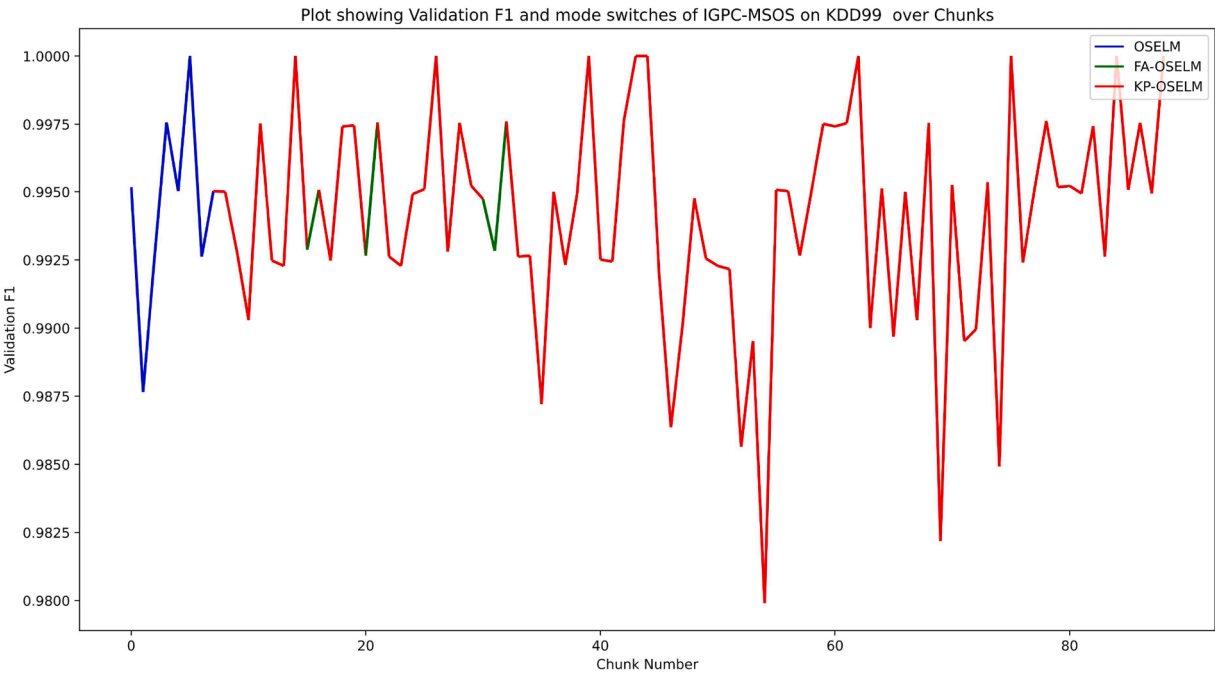


Fig. 9. F1-score and mode switching behavior of IGPC-MSOS on the KDD99 dataset across data chunks.

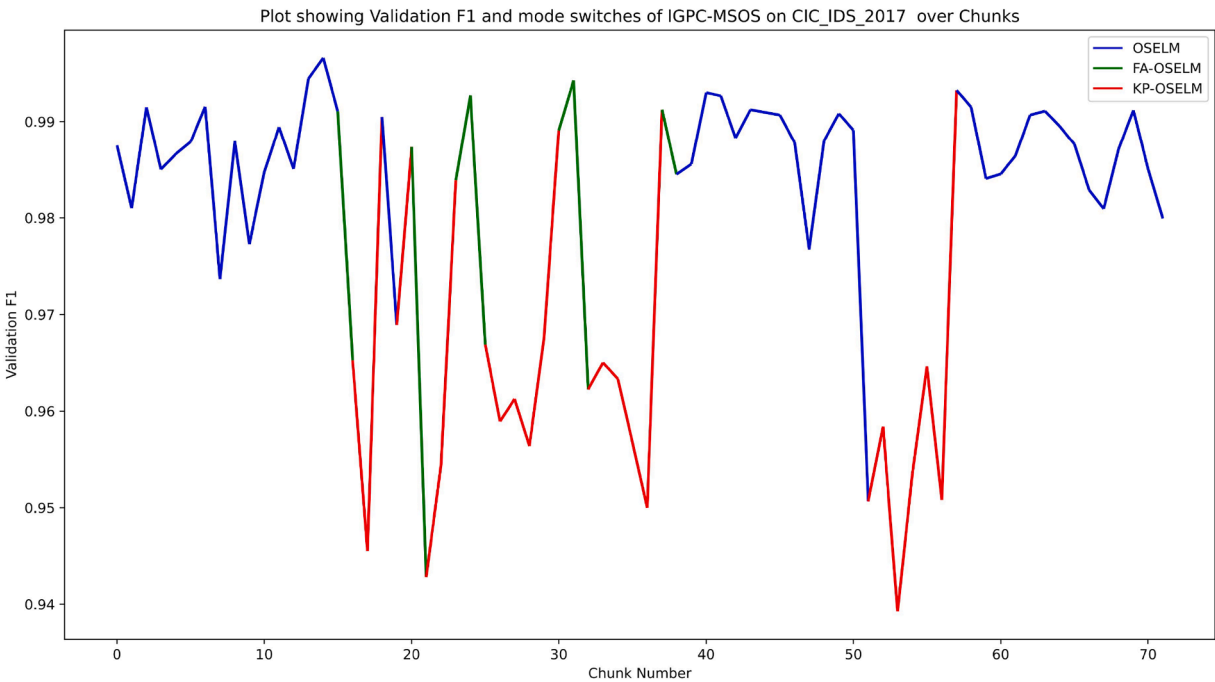


Fig. 10. F1-score and mode switching behavior of IGPC-MSOS on the CIC-IDS2017 dataset across data chunks.

Table 11
Summary of mode-switching dynamics across datasets.

Dataset	Dominant Behavior	Interpretation
KDD99	KP-OSELM dominant	Mild drift with recurring structure
CIC-IDS2017	OSELM dominant	Strong incremental drift suitability
CSE-CIC-IDS2018	Minimal switching, KP-OSELM	Near-perfect stability with trivial drift
IDS-ISCX2012	Rapid switching	Severe mixed and unstable drift
HIKARI2021	Dense cycling between modes	Distributed micro-drift and noise

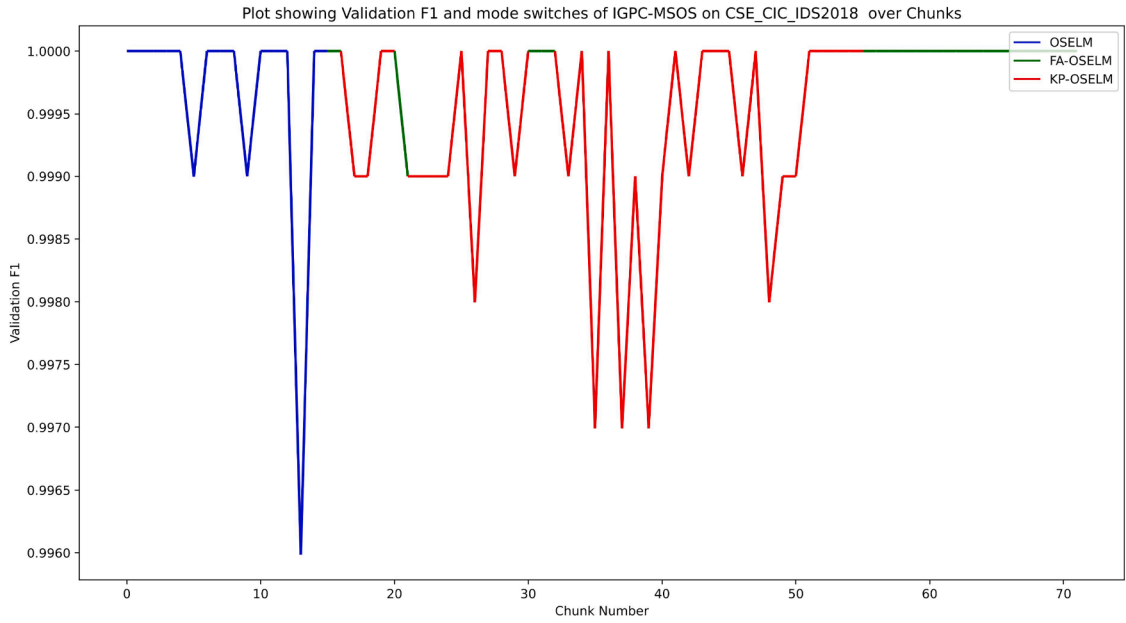


Fig. 11. F1-score and mode switching behavior of IGPC-MSOS on the CSE-CIC-IDS2018 dataset across data chunks.

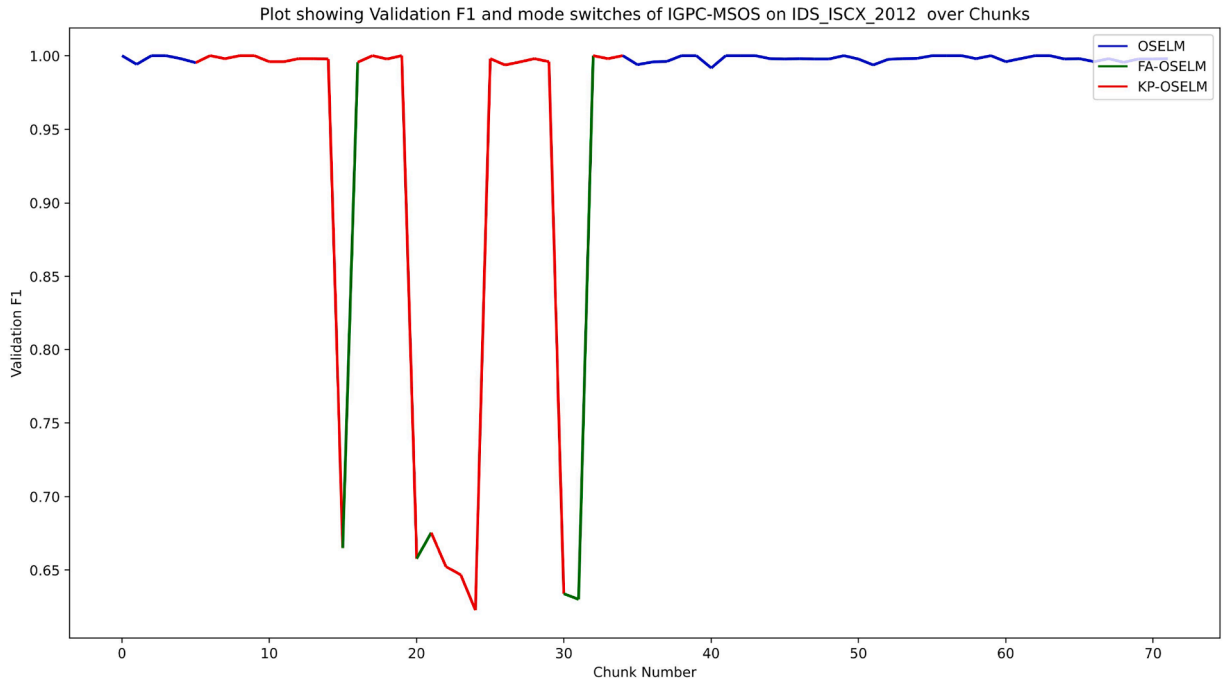


Fig. 12. F1-score and mode switching behavior of IGPC-MSOS on the IDS-ISCX2012 dataset across data chunks.

Statistical interpretation of mode switching

The switching patterns observed in Figs. 9–13 align with statistical findings reported in the performance comparisons. Datasets with dense or chaotic switching behavior, such as IDS-ISCX2012 and HIKARI2021, correspond to cases where IGPC-MSOS obtained statistically significant improvements over baselines ($p < .05$). Conversely, datasets where the controller converges toward a single mode—such as CIC-IDS2017 did not exhibit statistically significant differences relative to that single mode, confirming that the switching policy does not produce unnecessary volatility when one mode dominates. These statistical outcomes reinforce the correctness of the mode-switching logic and validate its ability to govern the learning dynamics effectively.

5.4. Ablation study: genetic programming vs. averaging ensemble

To validate the necessity of the Genetic Programming (GP) combiner and to address concerns about the benefit-to-complexity trade-off, we conducted an ablation study comparing GP-based ensemble aggregation with a simple averaging baseline. In the averaging method, classifier outputs are combined using uniform probability averaging, where $y_{avg} = \arg \max_c \frac{1}{|C|} \sum_{i \in C} P_i(c|x)$, with C denoting the set of base classifiers and $P_i(c|x)$ representing the probability assigned to class c by classifier i . In contrast, the GP combiner evolves optimized aggregation functions capable of capturing non-linear relationships between classifier outputs, as described in Section 3.3. Table 12 presents the F1-score comparison across all datasets and operational modes.

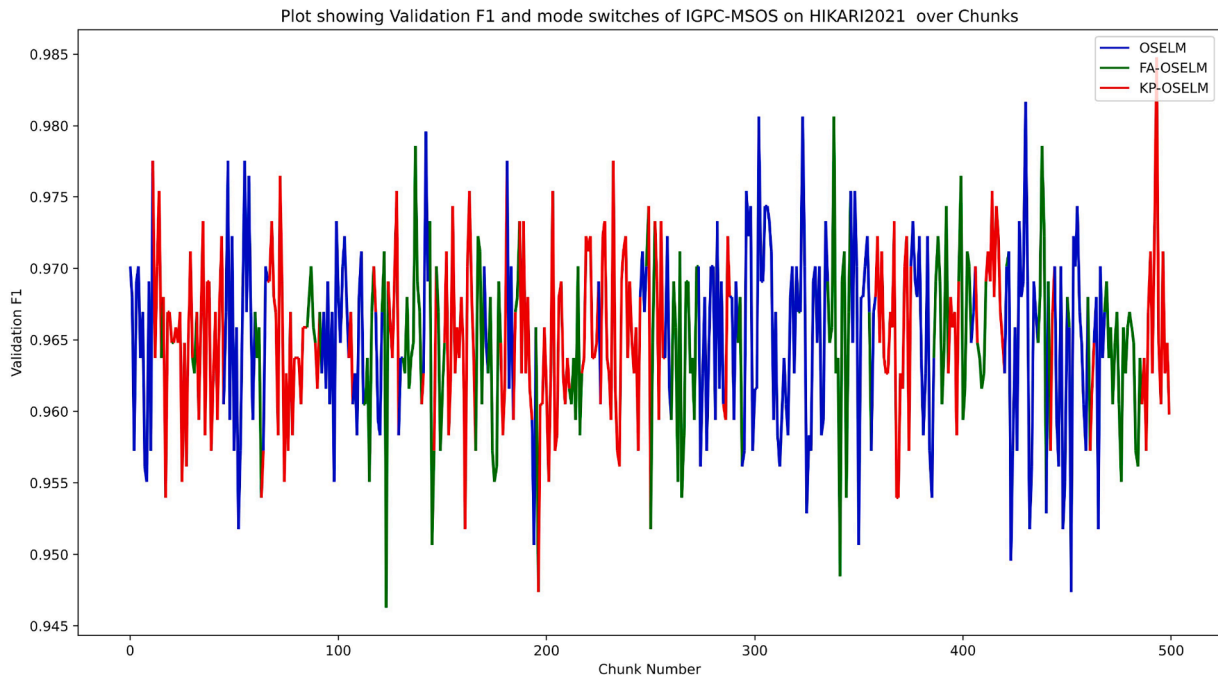


Fig. 13. F1-score and mode switching behavior of IGPC-MSOS on the HIKARI2021 dataset across data chunks.

Table 12

F1-score comparison: averaging method vs. genetic programming across all datasets and scenarios. Boldface values indicate cases where the Genetic Programming (GP) combiner outperforms the averaging method.

Dataset	Mode	Averaging	GP	Improvement (%)
CIC-IDS2017	GPC-OS	0.814200	0.981609	+20.56
	GPC-FOS	0.810354	0.951184	+17.38
	GPC-KOS	0.805369	0.917938	+13.98
	IGPC-MSOS (Ours)	0.807630	0.978868	+21.20
CSE-CIC-IDS2018	GPC-OS	0.999010	0.999113	+0.01
	GPC-FOS	0.999010	0.999911	+0.09
	GPC-KOS	0.999011	0.999018	+0.00
	IGPC-MSOS (Ours)	0.999011	0.999641	+0.06
HIKARI2021	GPC-OS	0.961121	0.961121	0.00
	GPC-FOS	0.961121	0.965128	+0.42
	GPC-KOS	0.961127	0.965128	+0.42
	IGPC-MSOS (Ours)	0.961124	0.965128	+0.42
IDS-ISCX2012	GPC-OS	0.673670	0.931297	+38.24
	GPC-FOS	0.673644	0.666113	-1.12
	GPC-KOS	0.673708	0.938760	+39.34
	IGPC-MSOS (Ours)	0.673798	0.959091	+42.34
KDD99	GPC-OS	0.943901	0.983622	+4.21
	GPC-FOS	0.944678	0.980331	+3.77
	GPC-KOS	0.945154	0.981084	+3.80
	IGPC-MSOS (Ours)	0.945104	0.994083	+5.18

The results demonstrate that Genetic Programming (GP) consistently outperforms or closely matches the averaging method, with particularly substantial improvements on drift-intensive datasets. The most pronounced gains are observed on IDS-ISCX2012, where GP achieves up to +42.34% F1 improvement for IGPC-MSOS compared to averaging. This dataset exhibits severe mixed drift patterns, under which the GP combiner's ability to dynamically weight classifier contributions becomes critical for maintaining robust detection performance.

On CIC-IDS2017, GP achieves consistent and meaningful improvements, ranging from approximately 14% to 21%, across all modes, confirming that evolutionary aggregation identifies superior combination strategies beyond uniform averaging. Similarly, on KDD99, GP yields

steady improvements in the range of approximately 3.8% to 5.2%, demonstrating that even in relatively stable environments, optimized ensemble aggregation remains beneficial.

For HIKARI2021 and CSE-CIC-IDS2018, where classifier performance is near saturation, GP produces only marginal gains. In HIKARI2021, GP yields small but positive improvements (approximately +0.42%) for most modes, while matching averaging exactly in the GPC-OS case. In CSE-CIC-IDS2018, GP improvements remain very limited (up to +0.09%), reflecting the constrained headroom for optimization in highly saturated settings. These results confirm that the added complexity of GP does not introduce harmful overhead when simpler aggregation suffices.

Minor degradations are observed only in isolated cases (e.g., GPC-FOS on IDS-ISCX2012), reflecting mode-specific instability rather than a limitation of GP itself. Importantly, IGPC-MSOS combined with GP consistently attains the highest overall performance across datasets, demonstrating that the integration of mode-switching with evolutionary optimization effectively mitigates individual mode weaknesses.

Table 13 summarizes the average improvement per dataset. Across all modes and datasets, Genetic Programming (GP) improves performance in 17 out of 20 cases, with an overall average improvement of +10.52%. These results justify the inclusion of GP-based ensemble optimization in the IGPC-MSOS framework, as the GP combiner provides the most substantial benefits in drift-intensive scenarios—particularly IDS-ISCX2012—where classifier stability is most severely challenged. At the same time, the limited gains observed on near-saturated datasets confirm that GP does not introduce adverse effects when simpler aggregation strategies are sufficient, supporting its robustness and practical suitability for adaptive intrusion detection.

5.5. Critical analysis of mode-switching effectiveness

To provide deeper insight into the mode-switching mechanism beyond frequency and dominance analyses, we conducted a critical evaluation of switch effectiveness, potential unnecessary transitions, and scenarios in which switching may degrade performance. Since IGPC-MSOS employs DDM (Drift Detection Method), an error-based approach that

Table 13
Summary of GP improvement over averaging by dataset.

Dataset	Avg. Improvement (%)	Max Improvement (%)	Modes Improved
IDS-ISCX2012	+29.70	+42.34	3/4
CIC-IDS2017	+18.28	+21.20	4/4
KDD99	+4.24	+5.18	4/4
CSE-CIC-IDS2018	+0.04	+0.09	3/4
HIKARI2021	+0.32	+0.42	3/4
Overall	+10.52	+42.34	17/20

Table 14
Mode-switching effectiveness analysis across datasets.

Dataset	Beneficial	Neutral	Detrimental	Total	Efficiency (%)
KDD99	0	6	0	6	100.00
CIC-IDS2017	1	13	1	15	93.33
CSE-CIC-IDS2018	0	7	0	7	100.00
HIKARI2021	15	57	13	85	84.71
IDS-ISCX2012	8	5	2	15	86.67
Total	24	88	16	128	87.50

operates without ground-truth drift labels, we evaluate switch effectiveness using empirical performance outcomes.

For each mode switch event s_i occurring at chunk t , we compute pre-switch and post-switch F1-scores over a 3-chunk window: $F1_{pre} = \frac{1}{3} \sum_{j=t-3}^{t-1} F1_j$ and $F1_{post} = \frac{1}{3} \sum_{j=t+1}^{t+3} F1_j$. Switches are then classified based on the performance differential $\Delta F1 = F1_{post} - F1_{pre}$ as beneficial ($\Delta F1 > 0.005$), neutral ($|\Delta F1| \leq 0.005$), or detrimental ($\Delta F1 < -0.005$). The efficiency rate is defined as the proportion of switches that were beneficial or neutral, calculated as (Beneficial + Neutral)/Total Switches $\times 100\%$.

Table 14 presents the switch classification results across all five benchmark datasets. Across all datasets, 87.50% of mode switches were either beneficial or neutral, indicating that the DDM-triggered switching mechanism generally makes appropriate adaptation decisions, with only 12.50% of switches (16 out of 128) resulting in measurable performance degradation.

The analysis reveals distinct behavioral patterns across datasets with varying drift characteristics. KDD99 and CSE-CIC-IDS2018 achieve 100% efficiency with zero detrimental switches, reflecting relatively stable drift patterns where the mode-switching controller operates conservatively. These datasets triggered only 6 and 7 switches, respectively, demonstrating that the minimum switching interval (Δt_{min}) effectively prevents unnecessary transitions when performance is stable.

In contrast, HIKARI2021 generated 85 total switches across its extended 500-chunk stream, substantially more than those of other datasets. Despite this high switching frequency, 84.71% of switches were non-detrimental. The 13 detrimental switches (15.29%) reflect the challenges posed by shallow, distributed micro-drift patterns that occasionally trigger exploratory mode transitions without sustained benefit. However, the cumulative impact of these detrimental switches remains limited, as evidenced by IGPC-MSOS achieving equivalent performance to the best single mode on this dataset (Table 9).

IDS-ISCX2012 demonstrates the highest proportion of beneficial switches (8 out of 15, 53.33%), reflecting genuine drift conditions that warranted mode adaptation. The 86.67% efficiency rate, combined with the substantial F1 improvement of IGPC-MSOS over single-mode variants on this dataset (+2.1% over GPC-KOS, Table 10), validates that mode-switching provides meaningful performance gains under challenging drift scenarios. CIC-IDS2017 achieves 93.33% efficiency with only 1 detrimental switch out of 15; however, as discussed in Section 5.2, the predominantly incremental drift structure of this dataset is well-suited to GPC-OS, which explains why IGPC-MSOS achieves slightly lower overall F1 than the optimal single mode.

Table 15
Benchmark comparison for the CIC-IDS-2017 dataset.

Metric	ATNN [5], 2024	FlowTransformer [63], 2024	IGPC-MSOS (Ours)
F1-score	0.7748	0.9940	0.9789
Accuracy	0.7757	0.9940	0.9796
Precision	0.7750	0.9932	0.9638
Recall	0.7757	0.9948	0.9950
Inference Time (s)	1.0730	0.023267	0.001705

Examining the 16 detrimental switches across all datasets reveals specific failure patterns. HIKARI2021 accounts for 13 of these cases, primarily caused by shallow micro-drift triggering the performance-stagnation clause ($\Delta \Phi < \epsilon$) without genuine drift, leading to exploratory switching to suboptimal modes. IDS-ISCX2012 contributes 2 detrimental switches resulting from transient error spikes misinterpreted as sustained drift, while CIC-IDS2017's single detrimental switch stems from a false positive DDM trigger caused by noise.

The mode-switching analysis demonstrates that although the mechanism achieves high overall efficiency (87.50%), certain scenarios reveal limitations. HIKARI2021's distributed, shallow drift patterns challenge the DDM error-based detection, occasionally triggering switches that do not improve performance. Conversely, the conservative behavior observed on stable datasets (KDD99 and CSE-CIC-IDS2018) with 100% efficiency validates that the switching parameters effectively prevent unnecessary transitions. The 12.50% detrimental switch rate represents an acceptable cost for the substantial gains achieved on drift-intensive datasets; on IDS-ISCX2012, despite 2 detrimental switches, IGPC-MSOS achieves the highest F1-score among all methods, confirming that the benefits of adaptive mode-switching outweigh occasional suboptimal transitions.

5.6. Comparison with state-of-the-art approaches

This section compares our Proposed Approach, IGPC-MSOS, against two established baselines across all five datasets: the FlowTransformer [63], a GPT-2-based deep sequential model, and the Adaptive Tree-like Neural Network (ATNN) [5], a lightweight adaptive method widely used in concept-drift research. Tables 15–19 summarize the per-dataset performance in terms of F1-score, accuracy, precision, recall, and inference time, and the consolidated comparison in Table 20 provides a cross-dataset perspective. The discussion highlights both predictive capability and computational efficiency—critical for real-time IDS.

5.6.1. CIC-IDS-2017 benchmark comparison

As shown in Table 15, the FlowTransformer achieves the highest F1-score and accuracy on CIC-IDS2017, reflecting its strength in modeling temporal dependencies in structured traffic. IGPC-MSOS, although slightly behind in overall F1, achieves the highest recall, demonstrating superior sensitivity to minority-class attack patterns. ATNN performs substantially worse in terms of F1 and accuracy and also incurs the highest inference latency (approximately 1.07 s), which severely limits its suitability for real-time deployment. By contrast, IGPC-MSOS maintains a very low inference time of about 1.7 ms, making it the fastest method and more than an order of magnitude faster than FlowTransformer (around 23 ms) while being several orders of magnitude faster than ATNN.

5.6.2. CSE-CIC-IDS2018 benchmark comparison

According to Table 16, all models achieve near-perfect performance on CSE-CIC-IDS2018. IGPC-MSOS attains the highest F1-score and perfect recall, outperforming both benchmarks in classification quality while preserving the lowest inference time (approximately 1.9 ms). FlowTransformer again incurs a substantial computational cost (about 24 ms), and ATNN, although competitive in F1 and accuracy, suffers

Table 16
Benchmark comparison for the CSE-CIC-IDS2018 dataset.

Metric	ATNN [5], 2024	FlowTransformer [63], 2024	IGPC-MSOS (Ours)
F1-score	0.99930	0.99927	0.99954
Accuracy	0.99928	0.99922	0.99908
Precision	0.99934	0.99861	0.99908
Recall	0.99928	0.99999	1.00000
Inference Time (s)	0.9000	0.023748	0.001869

Table 17
Benchmark comparison for the HIKARI2021 dataset.

Metric	ATNN [5], 2024	FlowTransformer [63], 2024	IGPC-MSOS (Ours)
F1-score	0.99930	0.99460	0.96513
Accuracy	0.99929	0.99458	0.93268
Precision	0.99932	0.99060	0.93268
Recall	0.99929	0.99864	1.00000
Inference Time (s)	1.1450	0.122805	0.001740

Table 18
Benchmark comparison for the IDS-ISCX2012 dataset.

Metric	ATNN [5], 2024	FlowTransformer [63], 2024	IGPC-MSOS (Ours)
F1-score	0.69155	0.97162	0.95909
Accuracy	0.69422	0.97185	0.94039
Precision	0.70123	0.97599	0.93865
Recall	0.69422	0.96736	0.99994
Inference Time (s)	0.6050	0.007273	0.000960

Table 19
Benchmark comparison for the KDD99 dataset.

Metric	ATNN [5], 2024	FlowTransformer [63], 2024	IGPC-MSOS (Ours)
F1-score	0.98376	0.99639	0.99408
Accuracy	0.98380	0.99638	0.99047
Precision	0.98374	0.99699	0.99178
Recall	0.98380	0.99580	0.99644
Inference Time (s)	0.9570	0.014546	0.001423

from very high latency (around 0.9 s), which makes it much less attractive for time-critical settings. Consequently, IGPC-MSOS offers the best trade-off between predictive performance and efficiency on this dataset.

5.6.3. HIKARI2021 benchmark comparison

Table 17 shows that HIKARI2021 strongly favors ATNN in terms of F1-score and accuracy, as it attains values close to 0.9993 on both metrics. However, this comes at the cost of a considerable inference time (approximately 1.15 s), which is prohibitive for real-time intrusion detection. IGPC-MSOS, while yielding lower F1 and accuracy, achieves perfect recall and the lowest inference latency (around 1.7 ms), providing a much more efficient yet still highly sensitive alternative. FlowTransformer lies between these two extremes, offering strong predictive performance but with substantially higher latency (about 123 ms) than IGPC-MSOS.

5.6.4. IDS-ISCX2012 benchmark comparison

As observed in Table 18, IDS-ISCX2012 presents substantial concept drift. FlowTransformer produces the highest overall F1-score due to its deep contextual modeling, whereas IGPC-MSOS provides the best recall by a wide margin (0.99994) while also achieving the fastest inference time (about 0.001 s). ATNN fails severely in classification accuracy and, in addition, exhibits the largest latency (approximately 0.61 s), making it the least suitable option in drift-heavy and time-sensitive environments. These results underscore the robustness and efficiency of IGPC-MSOS under strong drift conditions.

5.6.5. KDD99 benchmark comparison

As reported in Table 19, FlowTransformer attains the highest F1-score on KDD99, followed closely by IGPC-MSOS. The latter exceeds

Table 20
Overall benchmark summary across all datasets.

Dataset	Best F1	Best Recall	Fastest Inference
CIC-IDS2017	FlowTransformer	IGPC-MSOS	IGPC-MSOS
CSE-CIC-IDS2018	IGPC-MSOS	IGPC-MSOS	IGPC-MSOS
HIKARI2021	ATNN	IGPC-MSOS	IGPC-MSOS
IDS-ISCX2012	FlowTransformer	IGPC-MSOS	IGPC-MSOS
KDD99	FlowTransformer	IGPC-MSOS	IGPC-MSOS

FlowTransformer in recall and offers a substantially faster inference time (approximately 1.4 ms versus 14.5 ms). ATNN, in contrast, is both less accurate and dramatically slower (about 0.96 s), which precludes its use in latency-sensitive IDS scenarios. IGPC-MSOS therefore provides a better balance between detection capability and real-time feasibility.

Cross-dataset benchmark summary

Table 20 consolidates the benchmark landscape. FlowTransformer dominates the F1-score metric on several datasets, but remains computationally expensive. ATNN achieves the highest F1-score and accuracy on HIKARI2021, yet its inference times are consistently on the order of seconds, which makes it unsuitable for strict real-time constraints. IGPC-MSOS, by contrast, achieves the highest recall on every dataset and delivers the fastest inference across all benchmarks, with latencies below 2 ms. This combination of strong sensitivity to attacks and ultra-low latency positions IGPC-MSOS as the most practically viable solution among the three models.

Analytical discussion

Across all datasets, the FlowTransformer demonstrates strong static accuracy but suffers from substantial inference delays that limit its suitability for real-time intrusion detection. ATNN, while occasionally competitive in terms of F1-score and accuracy (notably on HIKARI2021 and CSE-CIC-IDS2018), exhibit inference times in the range of 0.6–1.15 s, which are incompatible with high-throughput or latency-critical environments. IGPC-MSOS provides the most balanced profile: it delivers the highest recall across all datasets, achieves state-of-the-art or near state-of-the-art F1-scores on several benchmarks, and maintains inference latencies between approximately 0.0010 and 0.0019 s. These latencies are three orders of magnitude lower than those of ATNN and substantially smaller than those of FlowTransformer.

The combined evidence, therefore, positions IGPC-MSOS as a robust, efficient, and highly adaptable model that is well aligned with the real-time demands of modern intrusion detection environments. It offers a clear advantage whenever both responsiveness and strong detection capability are required, particularly under pronounced concept drift.

Statistical significance of benchmark comparisons

The statistical analysis reveals that IGPC-MSOS significantly outperforms ATNN across all datasets ($p < 0.01$ or $p < 0.001$ except CSE-CIC-IDS2018 where differences are negligible). Compared to FlowTransformer, IGPC-MSOS demonstrates competitive performance with no significant difference on KDD99 and CSE-CIC-IDS2018, while FlowTransformer achieves significantly higher F1-scores on CIC-IDS2017, HIKARI2021, and IDS-ISCX2012. However, IGPC-MSOS consistently achieves superior recall and inference efficiency, making it more suitable for real-time, latency-critical deployments where false negatives carry high operational risk.

5.7. Comprehensive literature positioning (2021–2025)

To establish the broader context of our contributions within the rapidly evolving field of adaptive intrusion detection, Table 21 presents a comprehensive positioning of IGPC-MSOS against 20 state-of-the-art approaches published between 2021 and 2025. This analysis extends beyond the detailed comparisons with FlowTransformer and

Table 21

Comprehensive comparison with state-of-the-art IDS approaches (2021–2025). Symbol × indicates not evaluated on that dataset. AC = Accuracy, F1 = F1-Score, Re = Recall (all in %).

Ref.	Year	CIC17			CSE18			HIK21			ISCX			KDD		
		AC	F1	Re	AC	F1	Re	AC	F1	Re	AC	F1	Re	AC	F1	Re
CDDA-MD [6]	2025	98.24	98.64	×	×	×	×	×	×	×	×	×	×	×	×	×
ReCDA [4]	2024	96.30	96.30	×	×	×	×	×	×	×	×	×	×	×	×	×
MSANA [64]	2022	99.32	97.95	97.59	×	×	×	×	×	×	×	×	×	×	×	×
Recurrent-DL [66]	2022	98.00	96.00	97.00	×	×	×	×	×	×	×	×	×	89.00	35.00	50.00
ADWIN-SVM [15]	2022	98.50	98.50	97.10	×	×	×	×	×	×	×	×	×	98.20	99.90	98.10
SAW-WDA [25]	2023	98.10	88.00	90.50	×	×	×	×	×	×	×	×	×	×	×	×
DP-Means[71]	2025	85.00	83.80	83.10	×	×	×	×	×	×	×	×	×	×	×	×
CAEAID [72]	2025	×	×	×	91.20	91.30	92.09	×	×	×	×	×	×	×	×	×
BFO [69]	2024	×	×	×	92.10	92.70	92.50	×	×	×	×	×	×	×	×	×
NCM [3]	2022	×	×	×	96.50	94.20	92.10	×	×	×	×	×	×	×	×	×
OIFIDS [22]	2024	×	×	×	99.00	99.40	99.90	×	×	×	×	×	×	×	×	×
CNN, RNN[67]	2023	×	×	×	98.90	99.03	99.15	×	×	×	×	×	×	×	×	×
MLP-PSO [68]	2023	×	×	×	96.25	97.76	96.80	×	×	×	×	×	×	×	×	×
EHHO [73]	2022	×	×	×	94.20	95.86	92.48	×	×	×	×	×	×	×	×	×
GResNeSt[74]	2024	×	×	×	×	×	×	91.48	91.50	–	×	×	×	×	×	×
DT [65]	2025	×	×	×	×	×	×	77.60	75.64	77.60	×	×	×	×	×	×
Stack-Ens[65]	2025	×	×	×	×	×	×	79.00	78.11	79.00	×	×	×	×	×	×
Bag-Ens[65]	2025	×	×	×	×	×	×	75.50	74.22	75.50	×	×	×	×	×	×
OCNN-HMLSTM[70]	2021	×	×	×	×	×	×	×	×	×	95.33	97.61	94.77	×	×	×
OCNN [70]	2021	×	×	×	×	×	×	×	×	×	93.56	97.20	93.34	×	×	×
HMLSTM[70]	2021	×	×	×	×	×	×	×	×	×	94.99	95.38	90.75	×	×	×
EHID-SCA[75]	2024	×	×	×	×	×	×	×	×	×	×	×	×	98.69	98.76	98.90
IGPC-MSOS (Ours)		97.96	97.89	99.50	99.90	99.95	100.00	93.26	96.51	100.00	94.03	95.90	99.99	99.04	99.40	99.64

ATNN presented in Section 5.4.6, encompassing diverse methodological paradigms including concept drift-aware architectures [4,6], ensemble learning frameworks [64,65], deep learning adaptations [66,67], and optimization-based approaches [68,69].

Several observations emerge from this comprehensive comparison. First, dataset coverage heterogeneity remains a critical limitation across the literature. The majority of approaches evaluate on single datasets, with CICIDS2017 and CSE-CIC-IDS2018 being the most commonly adopted benchmarks, while HIKARI2021 and ISCX2012 receive substantially less attention. IGPC-MSOS uniquely provides consistent performance across all five benchmark datasets, demonstrating genuine cross-domain adaptability rather than dataset-specific optimization.

Second, recall performance superiority distinguishes IGPC-MSOS across operational scenarios. While several approaches achieve competitive accuracy and F1-scores on individual datasets, notably MSANA achieving 99.32% accuracy on CICIDS2017 [64] and ADWIN-SVM reaching 99.90% F1-score on KDD99 [15] IGPC-MSOS consistently delivers superior recall performance: 99.50% (CICIDS2017), 100% (CSE-CIC-IDS2018), 100.00% (HIKARI2021), 99.99% (ISCX2012), and 99.64% (KDD99). This recall dominance is particularly critical in cybersecurity contexts where false negatives (missed attacks) carry substantially higher operational risk than false positives.

Third, the performance-consistency trade-off reveals fundamental architectural differences. Approaches achieving exceptional performance on single datasets often demonstrate limited generalization when evaluated across multiple benchmarks. For instance, Recurrent-DL achieves 98.00% accuracy on CICIDS2017 but drops to 89.00% accuracy with only 50.00% recall on KDD99 [66], indicating brittleness under different concept drift patterns. In contrast, IGPC-MSOS maintains performance stability across all evaluated datasets, with accuracy ranging from 93.26% to 99.90% and recall consistently exceeding 99.50% in four out of five datasets.

Fourth, modern deep learning approaches demonstrate strong static performance but lack the explicit drift adaptation mechanisms that characterize IGPC-MSOS. CNN-RNN hybrid architectures achieve 99.03% F1-score on CSE-CIC-IDS2018 [67], and OIFIDS reaches 99.90% recall on the same dataset [22]. However, these approaches do not in-

corporate mode-switching capabilities, knowledge-preservation mechanisms, or transfer-learning strategies that enable IGPC-MSOS to handle diverse drift patterns systematically. IGPC-MSOS is the only framework that achieves both high recall and sub-2ms inference times across all datasets.

Finally, the ISCX2012 performance pattern warrants specific discussion. While IGPC-MSOS achieves 94.03% accuracy on this dataset—lower than OCNN-HMLSTM’s 95.33% [70]—our approach delivers substantially superior recall (99.99% vs. 94.77%), aligning with the cybersecurity principle that missed attacks are more operationally detrimental than false alarms. This performance characteristic reflects IGPC-MSOS’s explicit design for high-sensitivity detection under severe mixed drift conditions, which ISCX2012 exemplifies through its complex blend of sudden, gradual, and recurring concept drift patterns.

The comprehensive literature analysis confirms that IGPC-MSOS advances the state-of-the-art through consistent multi-dataset performance, superior recall sensitivity across diverse concept drift patterns, and an integrated mode-switching architecture that unifies transfer learning, knowledge preservation, and dynamic adaptation within a single framework. While the proposed approach demonstrates robust performance on widely used benchmark datasets, its evaluation is restricted to offline and replay-based data streams, and direct validation under live network traffic—where labels may be delayed, incomplete, or unavailable—remains an important direction for future investigation. Moreover, this work primarily focuses on adaptation effectiveness and performance robustness under concept drift; consequently, fine-grained explainability, such as attack-level decision interpretation and feature contribution analysis during drift, is intentionally treated as out of scope and constitutes a key avenue for future research.

6. Conclusion

This work introduced IGPC-MSOS, a unified and dynamically adaptive intrusion detection framework designed to address the limitations of traditional static and single-mode learning systems under concept drift. The proposed architecture integrates three complementary learners—OSELM, FA-OSELM, and KP-OSELM—within a mode-switching mechanism capable of responding intelligently to real, virtual, recurrent, and

mixed drift conditions. The system is further strengthened by a genetic programming-based combiner and a rigorous drift-detection strategy, enabling rapid adaptation to evolving network behaviors.

Extensive experiments conducted across five benchmark intrusion detection datasets demonstrated the robustness and versatility of IGPC-MSOS. The ablation study revealed that no single learner performs optimally across all drift settings: OSELM excels under incremental drift, KP-OSELM under recurrent drift, and FA-OSELM only under pronounced feature drift. IGPC-MSOS consistently exploited these strengths, achieving superior or comparable performance in four out of five datasets. Notably, the method achieved clear performance gains on the drift-intensive IDS-ISCX2012 dataset and maintained high stability across long data streams such as HIKARI2021. Furthermore, the ablation comparing genetic programming against simple averaging demonstrated that GP provides substantial improvements precisely under challenging drift conditions (+42.34% on IDS-ISCX2012, +10.52% average across all scenarios), justifying the architectural complexity through measurable performance gains.

The time-series mode-switching analysis confirmed that the controller operates as intended—activating OSELM in response to short-term changes, shifting to KP-OSELM under recurrent patterns, and briefly engaging FA-OSELM only when feature-level discrepancies emerge. The critical evaluation of switch effectiveness revealed that 87.50% of mode switches were beneficial or neutral, with detrimental switches primarily occurring under shallow micro-drift conditions. These dynamics demonstrate that IGPC-MSOS does not merely alternate between modes but learns a context-aware policy that aligns mode selection with underlying data characteristics.

Benchmark comparisons offered additional insight. While Flow-Transformer, a GPT-2-based architecture, achieved the highest static accuracy on certain datasets, it incurred substantial inference overhead. In contrast, IGPC-MSOS achieved recall values equal to or higher than those of both baselines while providing inference speeds up to two orders of magnitude faster. This highlights the method's potential suitability for real-time, high-throughput intrusion-detection environments, where latency and false negatives pose significant operational risk. The statistical evaluation confirmed these findings, demonstrating that IGPC-MSOS provides significant advantages over the benchmarks under drift-heavy conditions.

While IGPC-MSOS demonstrates strong performance across five diverse benchmark datasets, we acknowledge that these controlled experimental conditions do not fully capture enterprise-scale deployment challenges, including unlabeled streaming data, adversarial manipulation, and infrastructure heterogeneity. The framework's reliance on labeled data for DDM error computation, assumptions regarding consistent feature extraction pipelines, and potential vulnerability to adversarial evasion represent important limitations when transitioning from benchmark evaluation to operational deployment. Accordingly, the reported results should be interpreted as evidence of algorithmic capability rather than deployment-ready performance guarantees.

Overall, the results show that IGPC-MSOS effectively bridges the gap between accuracy, adaptability, and computational efficiency. Its capacity to learn when and how to switch between operational modes makes it well-suited for modern intrusion detection systems, where evolving threats, imbalanced traffic patterns, and structural changes in feature distributions challenge traditional static learning approaches. Future work will explore integrating deeper neural learners as optional modes within the switching framework, extending the method to multi-source streaming environments, and optimizing the switching logic through reinforcement learning to further enhance adaptability. Additionally, we plan to conduct pilot deployment studies with live network traffic, develop semi-supervised extensions to reduce label dependency, and perform systematic evaluation of adversarial robustness to bridge the gap between benchmark evaluation and operational readiness.

CRediT authorship contribution statement

Methaq A. Shyaa: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Noor Farizah Ibrahim:** Writing – review & editing, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Zurinahni Binti Zainol:** Writing – review & editing, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation; **Rosni Abdullah:** Writing – review & editing, Visualization, Supervision, Software, Project administration, Investigation, Formal analysis, Data curation; **Mohammed Anbar:** Writing – review & editing, Validation, Supervision, Methodology, Investigation, Formal analysis; **Laith Alzubaidi:** Writing – review & editing, Visualization, Validation, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation.

Data availability

Public datasets were used and cited within the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] S. Seth, S. Biswas, Concept drift detection and adaptation in streaming data: a comprehensive review, *Inf. Fusion* 91 (2024) 115–142.
- [2] S. Cai, Y. Zhao, Y. Hu, J. Wu, J. Wu, G. Zhang, C. Zhao, R.N.A. Sosu, CD-BTMSE: a concept drift detection model based on bidirectional temporal convolutional network and multi-stacking ensemble learning, *Knowl. Based Syst.* 294 (2024) 111681.
- [3] A. Kuppaa, N.-A. Le-Khac, Learn to adapt: robust drift detection in security domain, *Comput. Electr. Eng.* 102 (2022) 108239.
- [4] H. Yang, S. Zhang, J. Wang, RecDA: a novel recurring concept drift adaptation method for stream learning, *Inf. Sci.* 645 (2024) 119194.
- [5] W. Wen, H. Wang, J. Li, W. Wang, C. Wu, J. Zhang, Adaptive tree-like neural network: overcoming catastrophic forgetting to classify streaming data with concept drifts, *Knowl. Based Syst.* 285 (2024) 110284.
- [6] Y. Cai, X. Li, Y. Zhang, CDDA: a concept drift detection and adaptation framework for cybersecurity applications, *Fut. Gen. Comput. Syst.* 125 (2025) 779–795.
- [7] I.K. Thajeel, K. Samsudin, S.J. Hashim, F. Hashim, Dynamic feature selection model for adaptive cross site scripting attack detection using developed multi-agent deep q learning model, *J. King Saud Univ. Comput. Inf. Sci.*, 35 (6) (2023) 101490.
- [8] M.A. Shyaa, N.F. Ibrahim, Z. Zainol, R. Abdullah, M. Anbar, L. Alzubaidi, Evolving cybersecurity frontiers: a comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems, *Eng. Appl. Artif. Intell.* 137 (2024) 109143.
- [9] M.S. Noori, R.K.Z. Sahbudin, A. Sali, F. Hashim, Feature drift aware for intrusion detection system using developed variable length particle swarm optimization in data stream, *IEEE Access* 11 (2023) 128596–128617.
- [10] F. Uccello, M. Pawlicki, S. D'Antonio, R. Kozik, M. Choraś, An innovative approach to real-time concept drift detection in network security, in: *International Conference on Emerging Internet, Data & Web Technologies*, Springer, 2024, pp. 130–139.
- [11] Y. Zhang, J. Liu, R. He, S. Zhang, P.S. Yu, Adaptive random tree ensemble for evolving data streams under concept drift and class imbalance, *Knowl. Based Syst.* 284 (2025) 111347. <https://doi.org/10.1016/j.knsys.2024.111347>
- [12] M.A. Shyaa, Z. Zainol, R. Abdullah, M. Anbar, L. Alzubaidi, J. Santamaria, Enhanced intrusion detection with data stream classification and concept drift guided by the incremental learning genetic programming combiner, *Sensors* 23 (7) (2023) 3736.
- [13] G. Polino, F.S. Pisani, L. Pontieri, A GP-based ensemble classification framework for time-changing streams of intrusion detection data, *Soft comput.* 24 (23) (2020) 17541–17560.
- [14] L. Yang, D.M. Manias, A. Shami, Pwpae: an ensemble framework for concept drift adaptation in iot data streams, in: *2021 IEEE Global Communications Conference (GLOBECOM)*, IEEE, 2021, pp. 01–06.
- [15] P.V.N. Rajeswari, et al., Effective intrusion detection system using concept drifting data stream and support vector machine, *Concurrent. Comput. Prac. Exper.* 34 (21) (2022). <https://doi.org/10.1002/cpe.7118>
- [16] M. Jain, G. Kaur, Distributed anomaly detection using concept drift detection based hybrid ensemble techniques in streamed network data, *Cluster Comput.* 24 (3) (2021) 2099–2114.
- [17] G. Andresini, F. Penderbury, F. Pierazzi, C. Logliacci, A. Appice, L. Cavallaro, Insomnia: towards concept-drift robustness in network intrusion detection, in: *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security*, 2021, pp. 111–122.

- [18] M. Soltani, K. Khajavi, M. Jafari Siavoshani, A.H. Jahangir, A multi-agent adaptive deep learning framework for online intrusion detection, *Cybersecurity* 7 (1) (2024) 9.
- [19] D. Mulimani, S.G. Totad, P. Patil, S.V. Seeri, Adaptive ensemble learning with concept drift detection for intrusion detection, in: *Data Engineering and Intelligent Computing: Proceedings of ICICC 2020*, Springer, 2021, pp. 331–339.
- [20] Z. Shao, S. Yuan, Y. Wang, Adaptive online learning for IoT botnet detection, *Inf. Sci.* 574 (2021) 84–95.
- [21] O.A. Wahab, Intrusion detection in the IoT under data and concept drifts: online deep learning approach, *IEEE Internet Things J.* 9 (20) (2022) 19706–19716.
- [22] S.A. Elsaid, A. Binbusayis, An optimized isolation forest based intrusion detection system for heterogeneous and streaming data in the industrial internet of things (IIoT) networks, *Discover Appl. Sci.* 6 (9) (2024) 483.
- [23] K. Roshan, A. Zafar, Ensemble adaptive online machine learning in data stream: a case study in cyber intrusion detection system, *Int. J. Inf. Technol.* 16 (8) (2024) 5099–5112. Springer
- [24] N. Martindale, M. Ismail, D.A. Talbert, Ensemble-based online machine learning algorithms for network intrusion detection systems using streaming data, *Information* 11 (6) (2020) 315.
- [25] E. Alsuwat, S. Solaiman, H. Alsuwat, Concept drift analysis and malware attack detection system using secure adaptive windowing, *Comput. Mater. Continua* 75 (2) (2023).
- [26] H.S. Yazdi, A.G. Bafghi, et al., A drift aware adaptive method based on minimum uncertainty for anomaly detection in social networking, *Expert Syst. Appl.* 162 (2020) 113881.
- [27] M. Yang, J. Zhang, Data anomaly detection in the internet of things: a review of current trends and research challenges, *Int. J. Adv. Comput. Sci. Appl.* 14 (9) (2023).
- [28] L. Xu, X. Ding, H. Peng, D. Zhao, X. Li, ADTCD: an adaptive anomaly detection approach toward concept drift in IoT, *IEEE Internet Things J.* 10 (18) (2023) 15931–15942.
- [29] G. Werner, S.J. Yang, K. McConky, Near real-time intrusion alert aggregation using concept-based learning, in: *Proceedings of the 18th ACM International Conference on Computing Frontiers*, 2021, pp. 152–160.
- [30] M. Karimian, H. Beigy, Concept drift handling: a domain adaptation perspective, *Expert Syst. Appl.* 224 (2023) 119946.
- [31] Y. Chen, H.-L. Dai, Concept drift adaptation with continuous kernel learning, *Inf. Sci.* 670 (2024) 120649.
- [32] J. Chen, S. Yang, T. Gao, Y. Ying, T. Li, P. Li, Multi-type concept drift detection under a dual-layer variable sliding window in frequent pattern mining with cloud computing, *J. Cloud Comput.* 13 (1) (2024) 40.
- [33] O.A. Mahdi, E. Pardede, N. Ali, A hybrid block-based ensemble framework for the multi-class problem to react to different types of drifts, *Cluster Comput.* 24 (3) (2021) 2327–2340.
- [34] M. Sarnovsky, M. Kolarik, Classification of the drifting data streams using heterogeneous diversified dynamic class-weighted ensemble, *PeerJ Comput. Sci.* 7 (2021) e459.
- [35] S. Suryawanshi, A. Goswami, P. Patil, V. Mishra, Adaptive windowing based recurrent neural network for drift adaption in non-stationary environment, *J. Ambient Intell. Humaniz. Comput.* 14 (10) (2023) 14125–14139.
- [36] D.K. Talapula, A. Kumar, K.K. Ravulakollu, M. Kumar, A hybrid deep learning classifier and optimized key windowing approach for drift detection and adaption, *Decis. Anal. J.* 6 (2023) 100178.
- [37] T. Museba, F. Nelwamondo, K. Ouahada, A. Akinola, Recurrent adaptive classifier ensemble for handling recurring concept drifts, *Appl. Comput. Intell. Soft Comput.* 2021 (1) (2021) 5533777.
- [38] O. Wu, Y.S. Koh, G. Dobbie, T. Lacombe, Probabilistic exact adaptive random forest for recurrent concepts in data streams, *Int. J. Data Sci. Anal.* 13 (1) (2022) 17–32. Springer
- [39] O. Wu, Y.S. Koh, G. Dobbie, T. Lacombe, Nacre: proactive recurrent concept drift detection in data streams, in: *2021 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2021, pp. 1–8.
- [40] C.W. Chiu, L.L. Minku, A diversity framework for dealing with multiple types of concept drift based on clustering in the model space, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (3) (2020) 1299–1309.
- [41] A. Cano, B. Krawczyk, Kappa updated ensemble for drifting data stream mining, *Mach. Learn.* 109 (1) (2020) 175–218.
- [42] S. Bakhshi, P. Ghahramanian, H. Bonab, F. Can, A broad ensemble learning system for drifting stream classification, *IEEE Access* 11 (2023) 89315–89330. IEEE
- [43] K. Wang, J. Lu, A. Liu, Y. Song, L. Xiong, G. Zhang, Elastic gradient boosting decision tree with adaptive iterations for concept drift adaptation, *Neurocomputing* 491 (2022) 288–304.
- [44] G. Oliveira, L.L. Minku, A.L.I. Oliveira, Tackling virtual and real concept drifts: an adaptive gaussian mixture model approach, *IEEE Trans. Knowl. Data Eng.* 35 (2) (2021) 2048–2060. IEEE
- [45] S. Priya, R.A. Uthra, Deep learning framework for handling concept drift and class imbalanced complex decision-making on streaming data, *Complex Intell. Syst.* 9 (4) (2023) 3499–3515.
- [46] Y. Sun, M. Li, L. Li, H. Shao, Y. Sun, Cost-sensitive classification for evolving data streams with concept drift and class imbalance, *Comput. Intell. Neurosci.* 2021 (1) (2021) 8813806.
- [47] M. Li, H. Wang, L. Yang, Y. Liang, Z. Shang, H. Wan, Fast hybrid dimensionality reduction method for classification based on feature selection and grouped feature extraction, *Expert Syst. Appl.* 150 (2020) 113277.
- [48] Q. Li, Q. Xiong, S. Ji, Y. Yu, C. Wu, M. Gao, Incremental semi-supervised extreme learning machine for mixed data stream classification, *Expert Syst. Appl.* 185 (2021) 115591.
- [49] N. Liu, J. Zhao, Streaming data classification based on hierarchical concept drift and online ensemble, *IEEE Access* 11 (2023) 126040–126051. <https://doi.org/10.1109/ACCESS.2023.3327637>
- [50] B. Jiao, Y. Guo, D. Gong, Q. Chen, Dynamic ensemble selection for imbalanced data streams with concept drift, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (1) (2022) 1278–1291.
- [51] A. Abbasi, A.R. Javed, C. Chakraborty, J. Nebhen, W. Zehra, Z. Jalil, Elstream: an ensemble learning approach for concept drift detection in dynamic social big data stream learning, *IEEE Access* 9 (2021) 66408–66419. <https://doi.org/10.1109/ACCESS.2021.3076264>
- [52] H. Gálmeanu, R. Andonie, Concept drift adaptation with incremental-decremental SVM, *Appl. Sci.* 11 (20) (2021) 9644.
- [53] V. Patil, R. Kulkarni, Survey on extreme learning machines and its variants, *Artif. Intell. Rev.* 56 (2023) 12673–12705. <https://doi.org/10.1007/s10462-023-10573-2>
- [54] S. Wang, B. Zhang, S. Chen, J. Zhao, A review of extreme learning machine in neural computing, *Neural Comput. Appl.* 34 (2022) 15555–15572. <https://doi.org/10.1007/s00521-022-07006-0>
- [55] Y. Tang, M. Chen, J. Liu, An online sequential extreme learning machine for real-time intrusion detection, *IEEE Access* 9 (2021) 105384–105395. <https://doi.org/10.1109/ACCESS.2021.3100417>
- [56] J. Li, D. Zhang, L. Chen, G.-B. Huang, An improved online sequential extreme learning machine for classification of imbalanced data, *Neurocomputing* 275 (2018) 1102–1115. <https://doi.org/10.1016/j.neucom.2017.09.077>
- [57] A. AbuAlHajja, colleagues, Extreme learning machine for intrusion detection: a comprehensive survey, *Knowl. Based Syst.* 291 (2024) 111873. <https://doi.org/10.1016/j.knsys.2024.111873>
- [58] L. Wenxi, et al., Improved extreme learning machine for network intrusion detection, *Comput. Secur.* 124 (2023) 103030. <https://doi.org/10.1016/j.cose.2023.103030>
- [59] M. Tavallaee, E. Bagheri, W. Lu, A.A. Ghorbani, A detailed analysis of the KDD CUP 99 data set, in: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, IEEE, 2009, pp. 1–6.
- [60] A. Shiravi, H. Shiravi, M. Tavallaee, A.A. Ghorbani, Toward developing a systematic approach to generate benchmark datasets for intrusion detection, *Comput. Secur.* 31 (3) (2012) 357–374.
- [61] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, et al., Toward generating a new intrusion detection dataset and intrusion traffic characterization, *ICISSp* 1 (2018) 108–116.
- [62] A. Ferriyan, A.H. Thamrin, K. Takeda, J. Murai, Generating network intrusion detection dataset based on real and encrypted synthetic attack traffic, *Appl. Sci.* 11 (17) (2021) 7868.
- [63] L.D. Manocchio, S. Layeghy, W.W. Lo, G.K. Kulatilleke, M. Sarhan, M. Portmann, Flowtransformer: a transformer framework for flow-based network intrusion detection systems, *Expert Syst. Appl.* 241 (2024) 122564.
- [64] L. Yang, A. Shami, A multi-stage automated online network data stream analytics framework for IIoT systems, *IEEE Trans. Ind. Inf.* 19 (2) (2022) 2107–2116.
- [65] A. Araújo, D. Rodrigues, P. Leite, J. Gonçalves, Network intrusion detection system based on multiple datasets: machine learning approaches, in: *2025 13th International Symposium on Digital Forensics and Security (ISDFS)*, IEEE, 2025, pp. 1–5.
- [66] V. Ravi, R. Chaganti, M. Alazab, Recurrent deep learning-based feature fusion ensemble meta-classifier approach for intelligent network intrusion detection system, *Comput. Electr. Eng.* 102 (2022) 108156.
- [67] E.U.H. Qazi, M.H. Faheem, T. Zia, HDLIDS: hybrid deep-learning-based network intrusion detection system, *Appl. Sci.* 13 (8) (2023) 4921.
- [68] S. Alzughaibi, S. El Khediri, A cloud intrusion detection systems based on dnn using backpropagation and pso on the cse-cic-ids2018 dataset, *Appl. Sci.* 13 (4) (2023) 2276.
- [69] R. Rajeshwari, M.P. Anuradha, Intrusion detection using dynamic feature selection: an adaptive bacterial foraging method, *SN Comput. Sci.* 5 (6) (2024) 679.
- [70] P.R. Kanna, P. Santhi, Unified deep learning approach for efficient intrusion detection system using integrated spatial-temporal features, *Knowl. Based Syst.* 226 (2021) 107132.
- [71] C. Guo, X. Li, J. Cheng, S. Yang, H. Gong, Continual learning for intrusion detection under evolving network threats, *Fut. Internet* 17 (10) (2025) 456.
- [72] Z. Yin, H. Chen, H. Ma, T. Hu, L. Bai, CAEAD: an incremental contrast learning-based intrusion detection framework for IoT networks, *Comput. Netw.* 262 (2025) 111161.
- [73] Y. Xiao, C. Kang, H. Yu, T. Fan, H. Zhang, Anomalous network traffic detection method based on an elevated harris hawks optimization method and gated recurrent unit classifier, *Sensors* 22 (19) (2022) 7548.
- [74] D. Zhao, G. Ji, Y. Zhang, X. Han, S. Zeng, A network security situation prediction method based on SSA-GResnest, *IEEE Trans. Netw. Serv. Manage.* 21 (3) (2024) 3498–3510.
- [75] P. Chavan, H. Hanumanthappa, E.G. Satish, S. Manoli, S. Supreeth, S. Rohith, H.C. Ramaprasad, Enhanced hybrid intrusion detection system with attention mechanism using deep learning, *SN Comput. Sci.* 5 (5) (2024) 534.